

Java 24 — Vector API (Ninth Incubator)

Java Version: 24

Year: 2025

Feature: Vector API (Ninth Incubator)

JEP: 469

UniqueClassName: VectorAPI

Definition

The **Vector API** is an **incubating Java API** that enables developers to express **SIMD (Single Instruction, Multiple Data)** computations directly in Java.

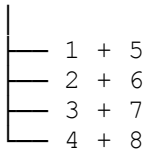
SIMD allows one CPU instruction to operate on **multiple values simultaneously**.

Instead of processing:

```
a[0] + b[0]
a[1] + b[1]
a[2] + b[2]
a[3] + b[3]
```

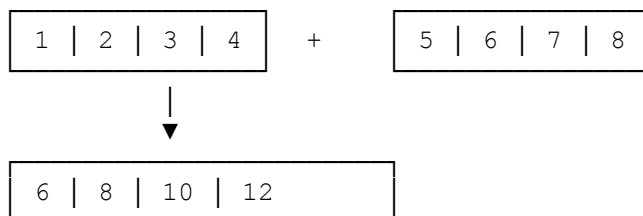
one at a time:

CPU



```
├── 1 + 5
├── 2 + 6
├── 3 + 7
└── 4 + 8
```

a vector instruction can conceptually process them together:



The goal is to make SIMD programming easier while allowing the JVM's JIT compiler to map vector operations to appropriate hardware instructions such as **AVX/AVX2/AVX-512** on supported x86 processors or **NEON/SVE** on supported Arm processors.

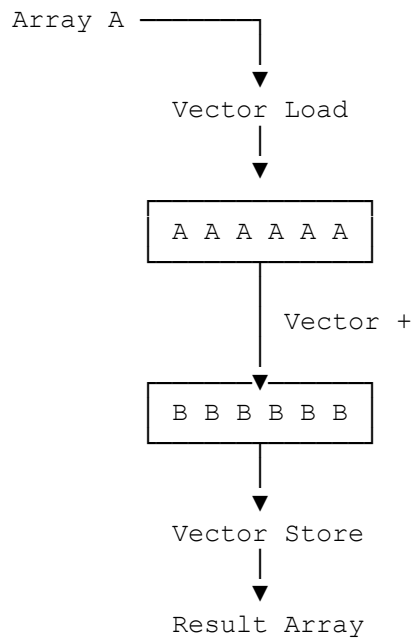
Why Vector API?

Traditional Java code:

```
for (int i = 0; i < length; i++) {  
    result[i] = a[i] + b[i];  
}
```

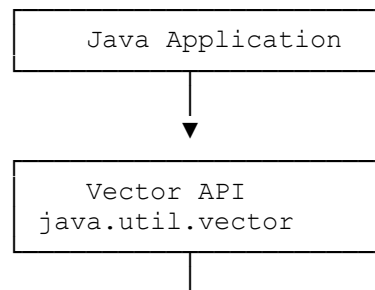
processes elements logically one at a time.

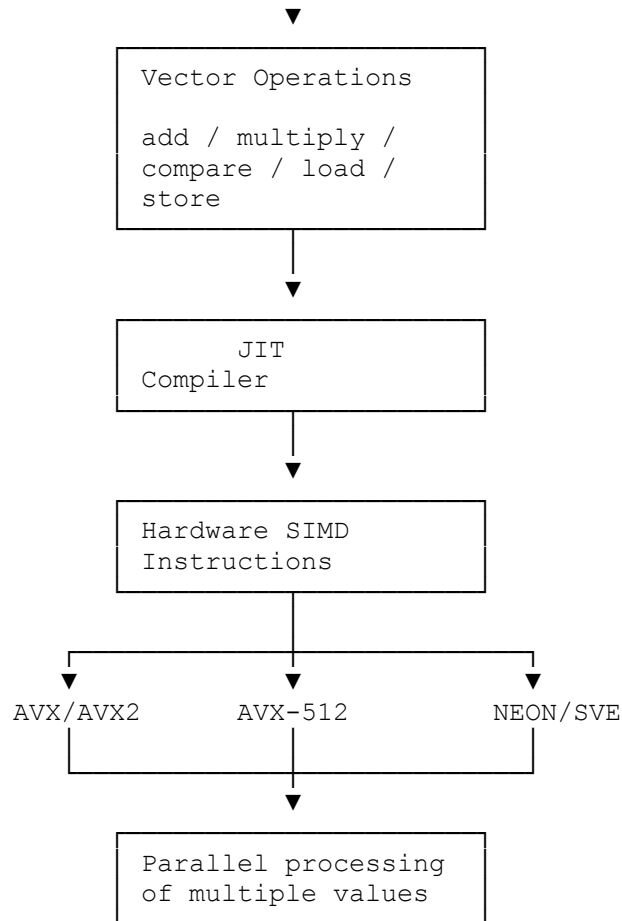
With the Vector API:



The JVM can then compile the vector operations into hardware SIMD instructions.

Flowchart





Java Code Example

Java 24's Vector API is still an **incubator feature**, so it is provided through the `jdk.incubator.vector` module.

Here is a simple vector addition example:

```
import jdk.incubator.vector.IntVector;
import jdk.incubator.vector.VectorSpecies;

public class VectorAPI {

    private static final VectorSpecies<Integer> SPECIES =
        IntVector.SPECIES_PREFERRED;

    public static void main(String[] args) {

        int[] a = {
            1, 2, 3, 4,
```

```

        5, 6, 7, 8
    };

    int[] b = {
        10, 20, 30, 40,
        50, 60, 70, 80
    };

    int[] result = new int[a.length];

    int i = 0;

    for (; i < SPECIES.loopBound(a.length);
        i += SPECIES.length()) {

        IntVector va =
            IntVector.fromArray(
                SPECIES, a, i);

        IntVector vb =
            IntVector.fromArray(
                SPECIES, b, i);

        IntVector vc =
            va.add(vb);

        vc.intoArray(result, i);
    }

    // Process remaining elements
    for (; i < a.length; i++) {
        result[i] = a[i] + b[i];
    }

    for (int value : result) {
        System.out.print(value + " ");
    }
}

```

Output:

11 22 33 44 55 66 77 88

Understanding the Code

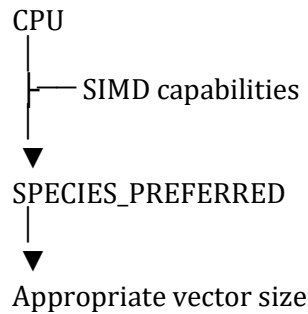
1. Select a Vector Species

```
private static final VectorSpecies<Integer> SPECIES =
```

```
IntVector.SPECIES_PREFERRED;
```

`SPECIES_PREFERRED` allows the JVM to select a vector shape suitable for the underlying hardware.

Conceptually:



This helps the same Java code adapt to different processors.

2. Load Values Into Vectors

```
IntVector va =  
    IntVector.fromArray(  
        SPECIES, a, i);
```

Instead of loading one integer:

`a[i]`

the vector operation loads multiple adjacent values:

a0	a1	a2	a3
----	----	----	----

The exact number of lanes depends on the selected vector species.

3. Vector Addition

```
IntVector vc = va.add(vb);
```

Conceptually:

A = [1, 2, 3, 4]

B = [10, 20, 30, 40]

Vector ADD

C = [11, 22, 33, 44]

One vector operation represents the element-wise addition.

4. Store the Result

vc.intoArray(result, i);

The vector is written back to the Java array:

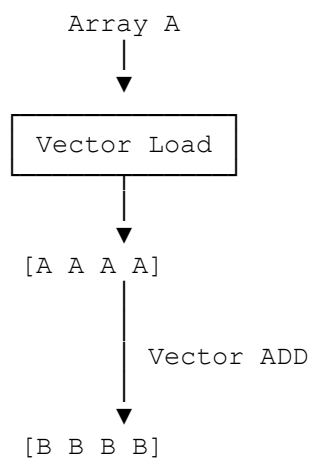
Vector

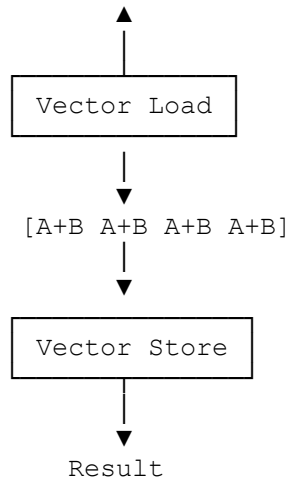
11	22	33	44
----	----	----	----



result[]

Complete Processing Flow





Scalar vs Vector Processing

Traditional Scalar Loop

```
for (int i = 0; i < a.length; i++) {  
    result[i] = a[i] + b[i];  
}
```

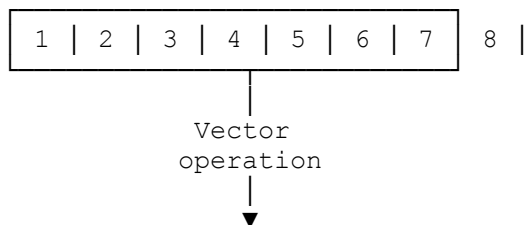
Conceptually:

1 operation
↓
2 operation
↓
3 operation
↓
4 operation

Vector API

```
va.add(vb);
```

Conceptually:



The hardware and JIT compiler determine how these operations are actually executed.

Compilation

Because the Vector API is an **incubator module**, compile with:

```
javac --add-modules jdk.incubator.vector VectorAPI.java
```

Run with:

```
java --add-modules jdk.incubator.vector VectorAPI
```

You may see an incubator-module warning when compiling or running.

Vector API Classes

Important classes include:

Class	Purpose
Vector	Base abstraction for vector computations
IntVector	Vector containing int values
LongVector	Vector containing long values
FloatVector	Vector containing float values
DoubleVector	Vector containing double values
VectorSpecies	Describes vector shape/type
VectorMask	Represents per-lane masks
VectorShuffle	Represents lane rearrangements

The package is:

`jdk.incubator.vector`

Example — Vector Multiplication

The same approach can be used for multiplication:

```
IntVector result =  
    vectorA.mul(vectorB);
```

Conceptually:

```
A = [2, 3, 4, 5]
```

```
B = [10, 20, 30, 40]
```

```
Vector MUL
```

```
C = [20, 60, 120, 200]
```

Example — Vector Comparison

Vectors can also be compared.

```
var mask =  
    va.compare(  
        VectorOperators.GT,  
        vb);
```

Conceptually:

```
A = [10, 20, 30, 40]
```

```
B = [15, 15, 35, 35]
```

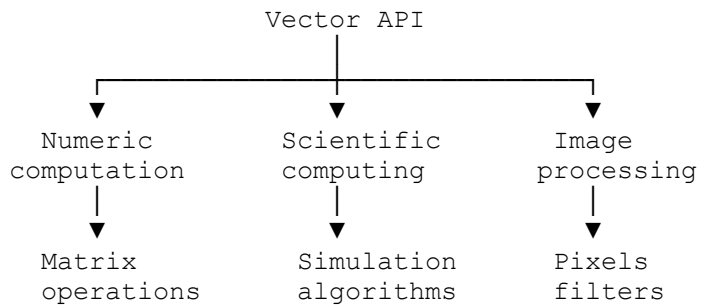
```
A > B
```

```
[false, true, false, true]
```

The resulting `VectorMask` can then be used for conditional vector operations.

Where Is Vector API Useful?

Vectorization is especially useful for data-parallel workloads:



Other examples include:

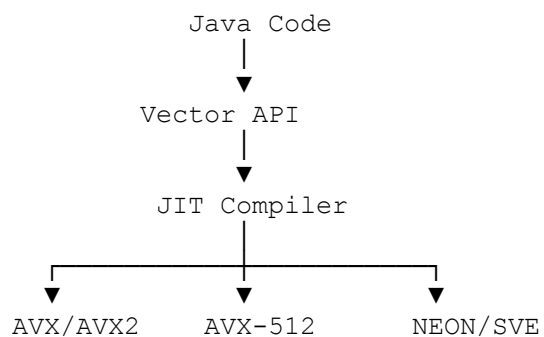
- Machine-learning calculations
- Signal processing
- Financial calculations
- Cryptographic primitives
- Image/video processing
- Scientific computing
- Large numerical arrays

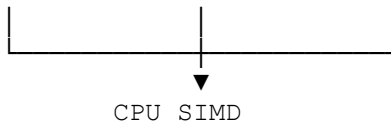
Vector API and Hardware

The Java programmer writes:

```
va.add(vb);
```

rather than directly writing CPU-specific instructions.



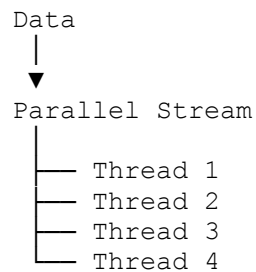


This is one of the major benefits of the API: **portable Java source with hardware-specific optimization performed by the JVM.**

Vector API vs Parallel Streams

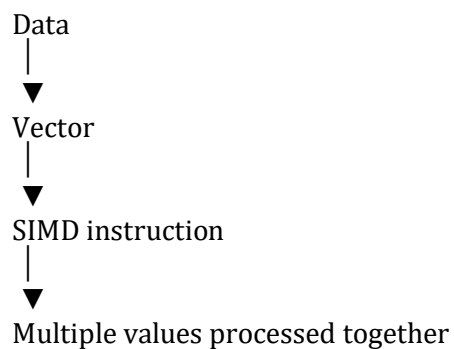
These are different approaches.

Parallel Stream



Parallelism comes primarily from **multiple threads/cores.**

Vector API



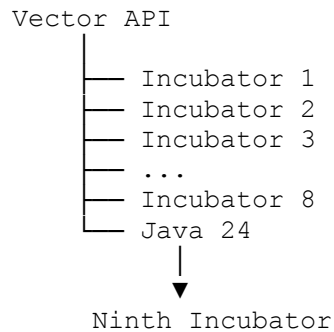
Vectorization happens at the **CPU instruction level.**

They can potentially be combined in suitable workloads, but doing so indiscriminately can hurt performance.

Ninth Incubator

The phrase "**Ninth Incubator**" is important.

The Vector API has gone through multiple incubator iterations as the Java platform team refined its design.



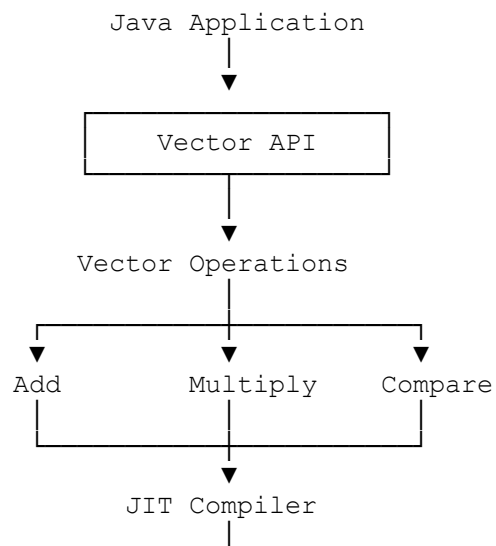
This means the API was **not yet a final permanent Java SE API in Java 24**.

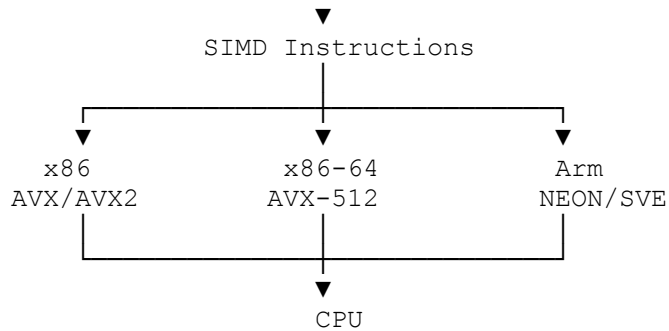
The package remains:

`jdk.incubator.vector`

rather than a standard permanent API package.

Overall Architecture





Key Takeaways

Property	Details
Java Version	24
Year	2025
JEP	469
Status	Ninth Incubator
Package	jdk.incubator.vector
Main purpose	SIMD/data-parallel computation
Core concept	One operation on multiple values
Main classes	Vector, IntVector, FloatVector, DoubleVector, etc.
Hardware optimization	Performed by JIT/compiler
CPU examples	AVX, AVX2, AVX-512, NEON, SVE
Java syntax	No new language syntax
Compilation	--add-modules jdk.incubator.vector
Best suited for	Numerical and data-parallel workloads

One-line definition

Java 24's Vector API (JEP 469), in its ninth incubator, provides a portable Java API for expressing SIMD computations so that the JIT compiler can map operations on multiple data elements to efficient hardware vector instructions.

Java 24 — Remove Non-Generational ZGC

Java Version: 24

Year: 2025

Feature: Remove Non-Generational ZGC

JEP: 491?

Correction: The removal of non-generational ZGC is associated with **JEP 490 — Remove the Non-Generational Mode of ZGC**, delivered in **JDK 24**.

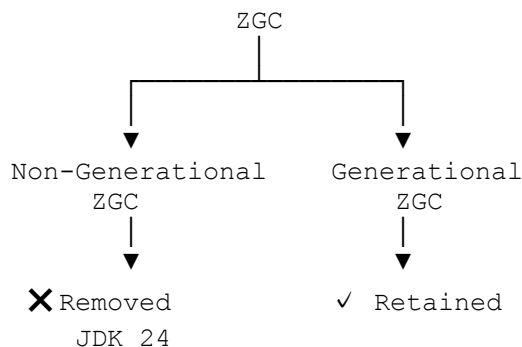
UniqueClassName: RemoveNonGenerationalZGC

Definition

Remove Non-Generational ZGC is a **Java 24 JVM change** that removes ZGC's old **non-generational mode**.

ZGC is a low-latency garbage collector designed to keep GC pauses extremely short, even with very large heaps.

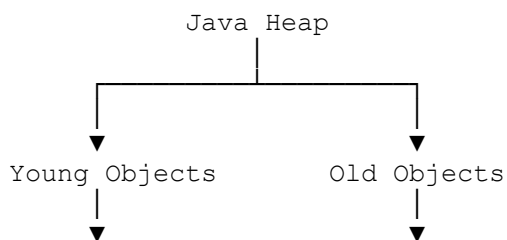
Before Java 24, ZGC could operate in two modes:



Starting with **JDK 24**, ZGC is **generational only**.

What Is Generational ZGC?

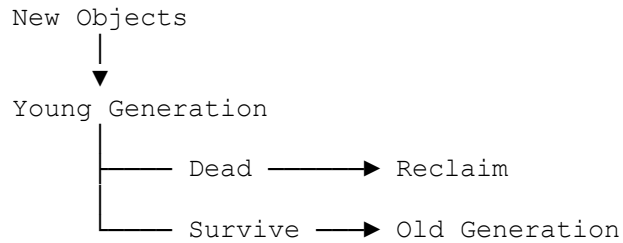
Generational garbage collection divides objects according to their age.



Short-lived

Long-lived

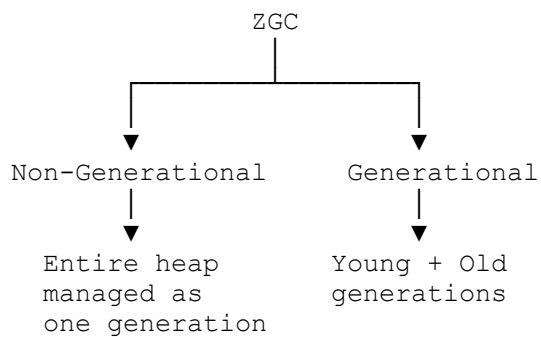
The design is based on the observation that most newly created objects become unreachable relatively quickly.



Generational ZGC can therefore focus collection effort where garbage is most likely to exist.

Before Java 24

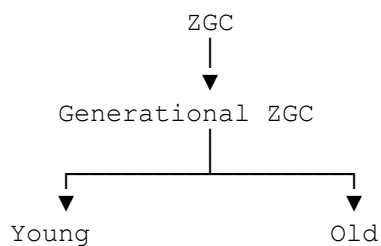
ZGC supported:

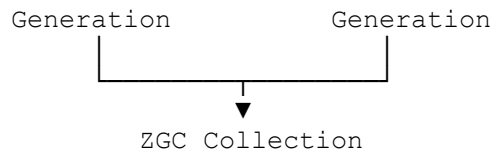


A user could explicitly select the old non-generational mode.

Java 24

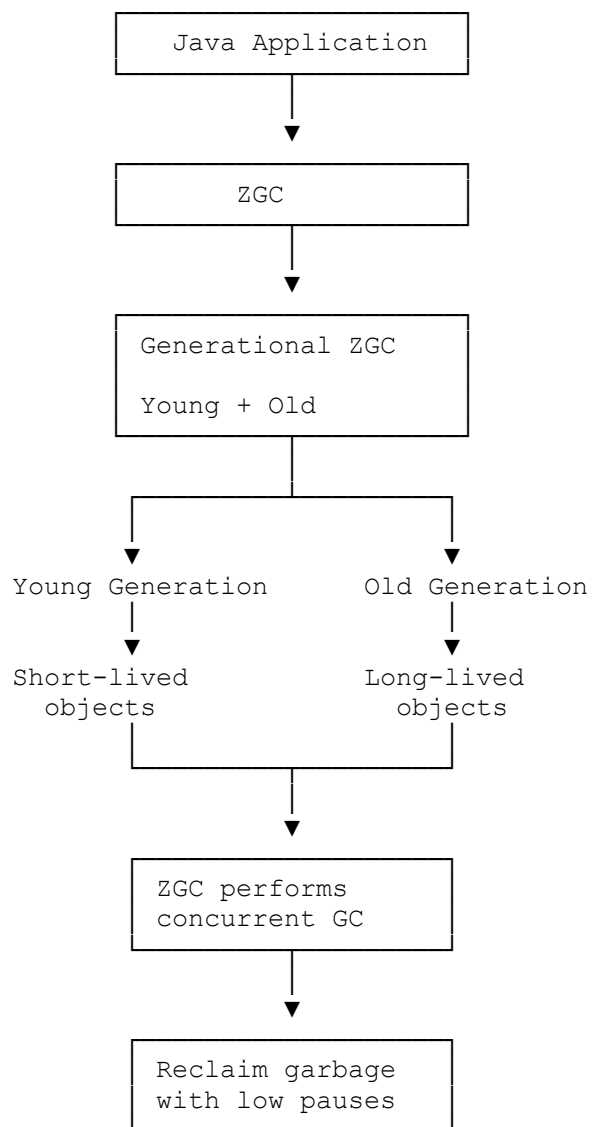
Java 24 simplifies the model:





The old non-generational mode is gone.

Flowchart

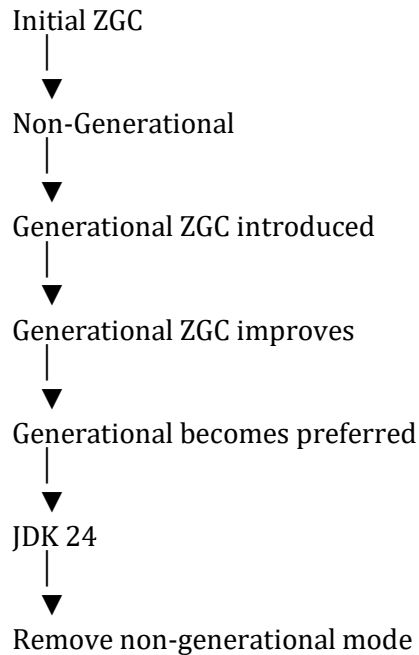


Why Was Non-Generational ZGC Removed?

The main reason is that the **generational implementation became the preferred ZGC architecture**.

Generational ZGC was introduced earlier as an experimental/preview mode and subsequently matured.

Conceptually:



Maintaining two substantially different ZGC modes also created additional implementation and testing complexity.

Removing the old mode allows ZGC development to concentrate on the generational implementation.

Java Code Example

This feature **does not change Java source-code syntax**.

An ordinary Java program can use ZGC:

```
public class RemoveNonGenerationalZGC {  
    public static void main(String[] args) {
```

```
System.out.println(
    "Java Version: "
    + System.getProperty("java.version")
);

for (int i = 0; i < 1_000_000; i++) {

    // Create short-lived objects
    byte[] data = new byte[1024];

    data[0] = (byte) i;
}

System.out.println(
    "Application completed."
);
}
}
```

The application code doesn't explicitly invoke ZGC.

The JVM controls garbage collection.

Running With ZGC

In Java 24:

```
java -XX:+UseZGC RemoveNonGenerationalZGC
```

This uses ZGC.

Because Java 24 has removed the non-generational ZGC mode, ZGC uses its generational implementation.

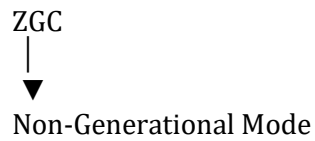
You can enable GC logging:

```
java -XX:+UseZGC \
-Xlog:gc \
RemoveNonGenerationalZGC
```

Old Configuration

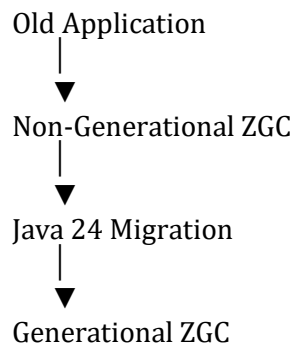
Applications using older Java releases might have explicitly selected non-generational ZGC.

Conceptually:



Such configurations need to be reviewed when migrating to Java 24.

The migration principle is:

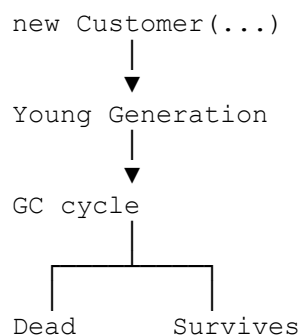


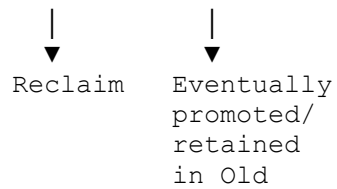
Generational Object Lifecycle

Consider:

```
Customer customer =  
    new Customer("Anurag");
```

The object is initially young:





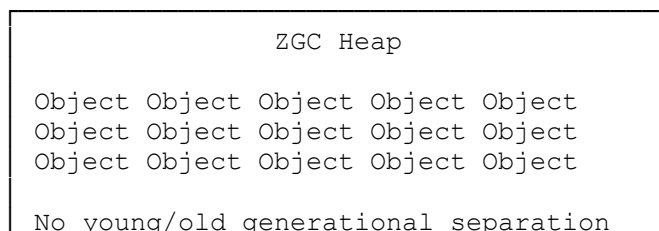
Short-lived objects can be reclaimed efficiently without repeatedly treating the entire heap in the same way.

Non-Generational vs Generational ZGC

Aspect	Non-Generational ZGC	Generational ZGC
Java 23 and earlier	Available	Available
Java 24	Removed	Used
Generations	No	Young + Old
Object age considered	Less explicitly	Yes
Focus	Entire heap	Young/old distinction
Main benefit	Simpler historical model	Better efficiency for typical object lifetimes
Java source changes	None	None

Heap Visualization

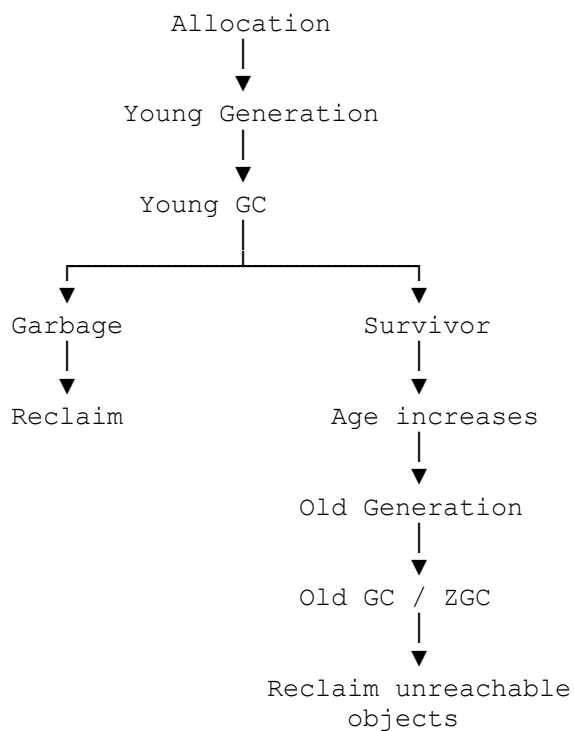
Old Non-Generational ZGC



Java 24 Generational ZGC

ZGC Heap
Young Generation New objects / short-lived objects
Old Generation Long-lived / surviving objects

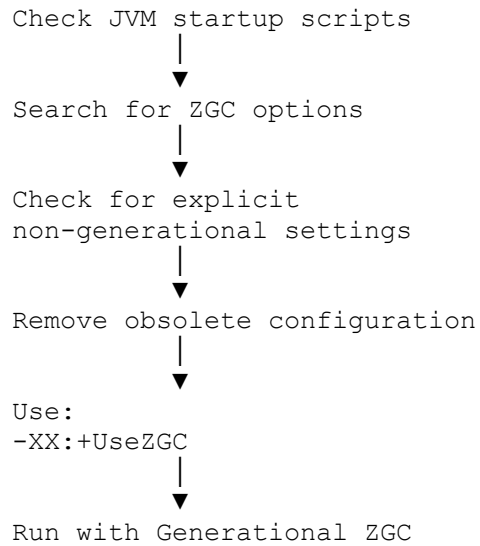
GC Processing



ZGC performs much of its work **concurrently** with application execution, maintaining its low-latency characteristics.

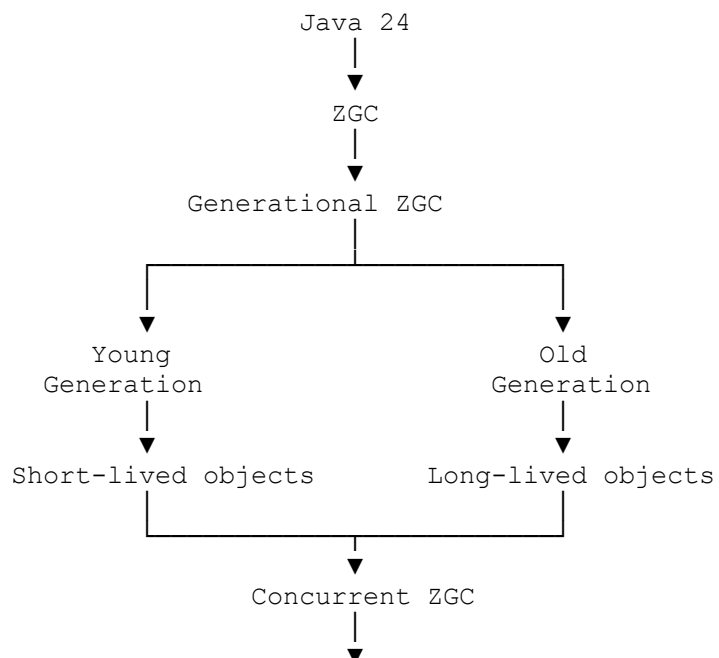
Important Migration Point

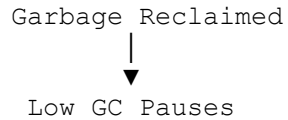
If you are upgrading an application from an earlier JDK to Java 24:



The key change is therefore mostly **operational/JVM configuration**, not application source code.

Overall Architecture





Key Takeaways

Property	Details
Java Version	24
Year	2025
JEP	490
Feature	Remove the Non-Generational Mode of ZGC
Type	JVM / Garbage Collector change
Removed	Non-generational ZGC
Retained	Generational ZGC
Generations	Young + Old
Java syntax changes	None
Application code changes	Normally none
Main motivation	Simplify ZGC and focus on generational implementation
GC goal	High throughput with very low pause times

One-line definition

Java 24's JEP 490 removes ZGC's non-generational mode, making Generational ZGC the sole ZGC mode and allowing the JVM to focus on collecting young and old objects according to their expected lifetimes while maintaining ZGC's low-latency design.

Java 24 — Synchronize Virtual Threads without Pinning

Java Version: 24

Year: 2025

Feature: Synchronize Virtual Threads without Pinning

JEP: 491

UniqueClassName: VirtualThreadSynchronization

Definition

Synchronize Virtual Threads without Pinning is a **Java 24 JVM improvement** that changes how **synchronized blocks and methods interact with virtual threads**.

Before Java 24, a virtual thread could become **pinned to its carrier platform thread** when it entered certain synchronized regions. While pinned, the JVM could not freely unmount the virtual thread from that carrier thread during a blocking operation.

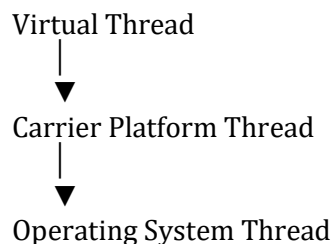
Java 24 changes the JVM implementation so that **virtual threads can block inside synchronized constructs without being pinned to their carrier threads in the situations addressed by JEP 491**.

This makes existing synchronization code much more compatible with virtual threads.

What Is Virtual-Thread Pinning?

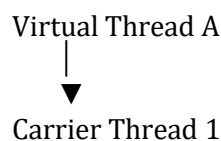
A virtual thread is not permanently tied to an operating-system thread.

Normally:



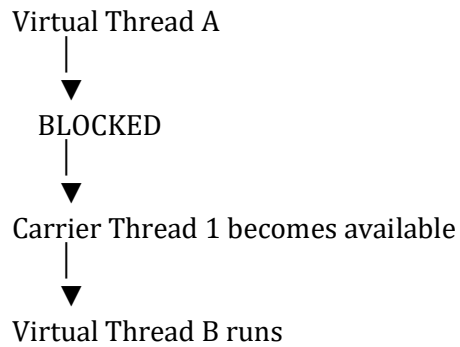
The JVM can **mount** and **unmount** virtual threads from carrier threads.

For example:





When Virtual Thread A blocks, the JVM can potentially unmount it:



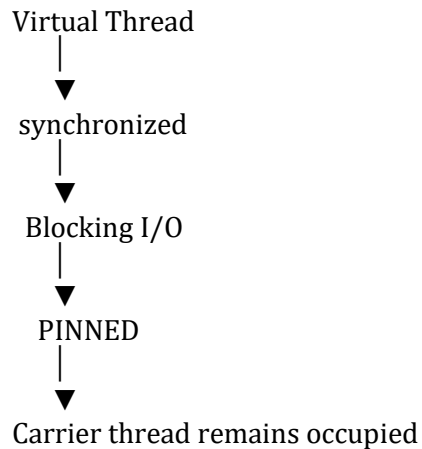
This is one of the important scalability characteristics of virtual threads.

The Problem Before Java 24

Certain uses of `synchronized` could prevent this unmounting.

```
synchronized (lock) {  
    blockingOperation();  
}
```

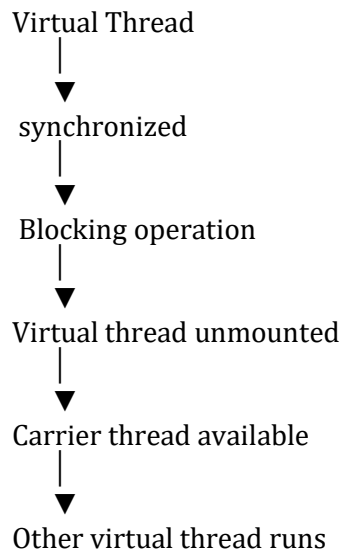
Conceptually:



If many virtual threads became pinned in this way, the carrier-thread pool could become less effective.

Java 24 Solution

JEP 491 changes the JVM so that virtual threads can generally **park/block while executing synchronized code without pinning their carrier thread**.



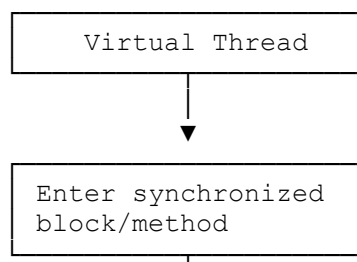
The important point is that the **Java synchronization syntax does not change**.

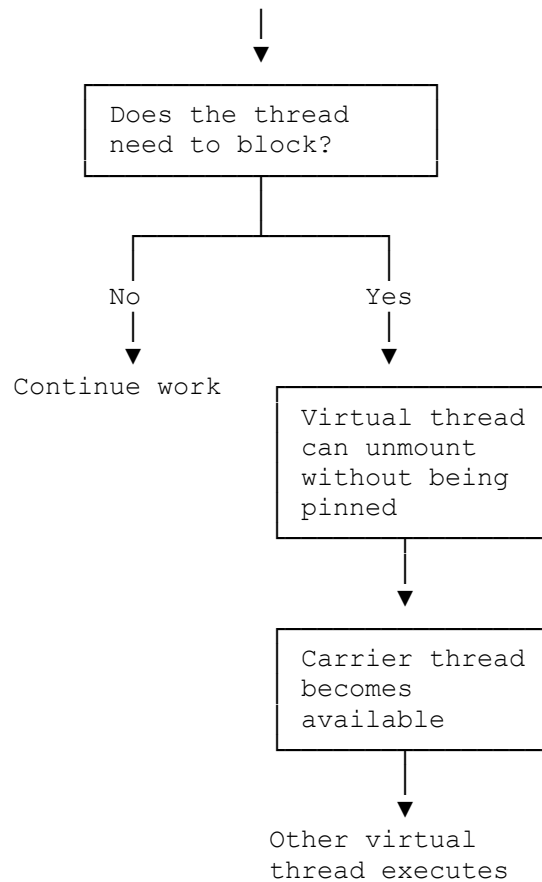
Existing code such as:

```
synchronized (lock) {  
    // protected code  
}
```

can continue to be used.

Flowchart





Java Code Example

The Java source code is straightforward.

```
public class VirtualThreadSynchronization {  
    private static final Object LOCK =  
        new Object();  
  
    public static void main(String[] args)  
        throws InterruptedException {  
  
        Thread virtualThread =  
            Thread.ofVirtual().start(() -> {  
  
            synchronized (LOCK) {  
  
                System.out.println(  
                    "Entered synchronized block"  
                );  
  
                try {  

```

```

        Thread.sleep(1000);
    } catch (InterruptedException e) {
        Thread.currentThread()
            .interrupt();
    }

    System.out.println(
        "Leaving synchronized block"
    );
}
});

virtualThread.join();

System.out.println(
    "Main thread completed."
);
}
}

```

Possible output:

```

Entered synchronized block
Leaving synchronized block
Main thread completed.

```

The important part is:

```

synchronized (LOCK) {
    Thread.sleep(1000);
}

```

In older Java implementations, blocking inside certain synchronized regions could pin the virtual thread.

With Java 24's JEP 491 changes, the JVM can handle this synchronization without the same carrier-thread pinning problem.

Multiple Virtual Threads Example

The benefit becomes easier to understand with many virtual threads.

```

public class VirtualThreadSynchronization {

    private static final Object LOCK =
        new Object();

    public static void main(String[] args)

```

```

        throws InterruptedException {

    for (int i = 1; i <= 5; i++) {

        final int id = i;

        Thread.startVirtualThread(() -> {

            synchronized (LOCK) {

                System.out.println(
                    "Virtual Thread "
                    + id
                    + " entered lock."
                );

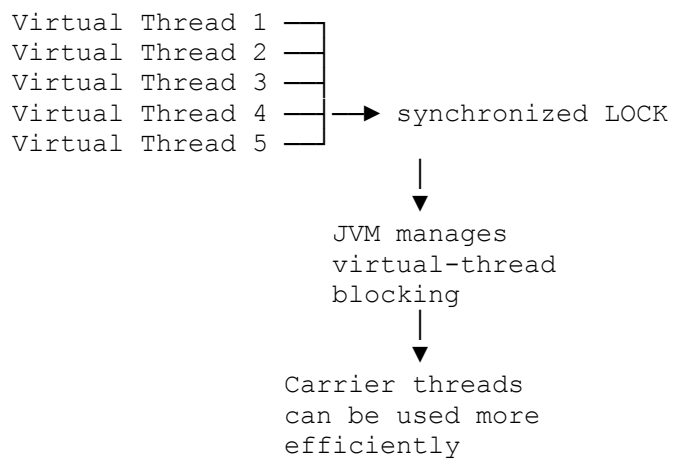
                try {
                    Thread.sleep(500);
                } catch (InterruptedException e) {
                    Thread.currentThread()
                        .interrupt();
                }

                System.out.println(
                    "Virtual Thread "
                    + id
                    + " leaving lock."
                );
            }
        });
    }

    Thread.sleep(3000);
}

```

Conceptually:



Because all threads use the same lock, the threads still have to obey **mutual exclusion**. JEP 491 does **not** make a synchronized block execute concurrently.

Synchronization vs Pinning

These are two different concepts.

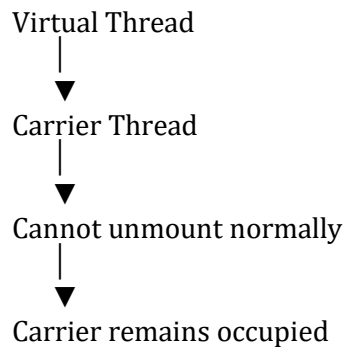
Synchronization

Ensures that only one thread at a time enters a particular monitor-protected region.

```
Thread A → LOCK → Critical Section
Thread B → WAIT
Thread C → WAIT
```

Pinning

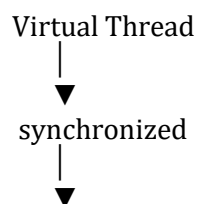
Means a virtual thread cannot be freely unmounted from its carrier thread.

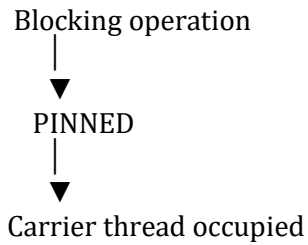


Java 24 addresses the **pinning behavior**, not the purpose of synchronization.

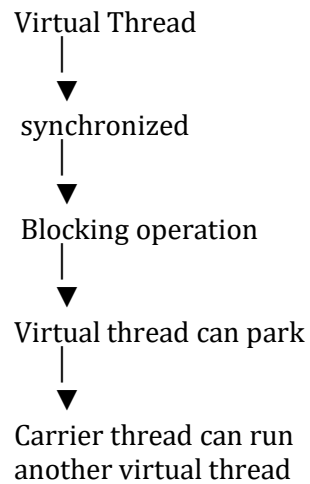
Before Java 24 vs Java 24

Before





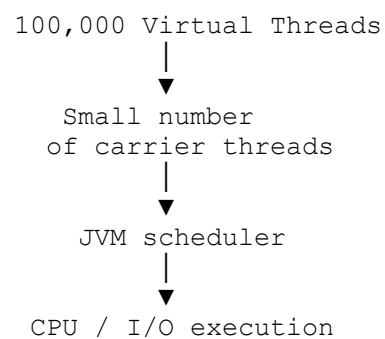
Java 24



Why This Matters

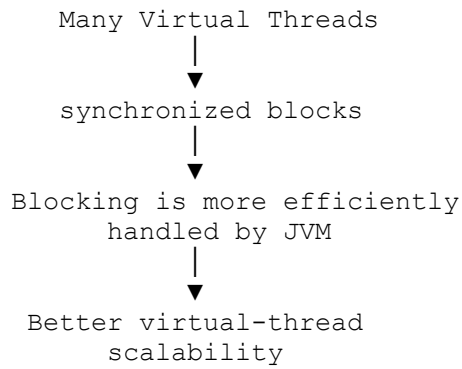
Virtual threads are intended to support very large numbers of concurrent tasks.

For example:



If synchronization unnecessarily pinned carrier threads, scalability could suffer.

Java 24 improves this situation:



Important: `synchronized` Is Still a Lock

JEP 491 does **not** eliminate locking.

For example:

```
synchronized (LOCK) {  
    updateSharedState();  
}
```

still guarantees mutual exclusion.

If two virtual threads execute:

```
Thread A → synchronized → ENTER  
Thread B → synchronized → WAIT  
Thread C → synchronized → WAIT
```

only one can own the monitor at a time.

After Thread A leaves:

```
Thread A  
|  
▼  
Release LOCK  
|  
▼  
Thread B enters
```

So the change is about **how the JVM manages virtual threads around synchronization**, not about changing Java's synchronization semantics.

ReentrantLock Is Still Useful

Before Java 24, developers were sometimes advised to use ReentrantLock instead of synchronized in virtual-thread applications to avoid pinning.

For example:

```
private final ReentrantLock lock =
    new ReentrantLock();

lock.lock();

try {
    // protected work
} finally {
    lock.unlock();
}
```

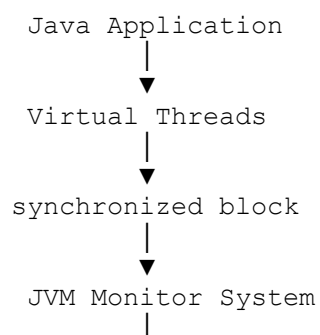
With Java 24's JEP 491 changes, the need to replace ordinary synchronized blocks **solely to avoid virtual-thread pinning** is substantially reduced.

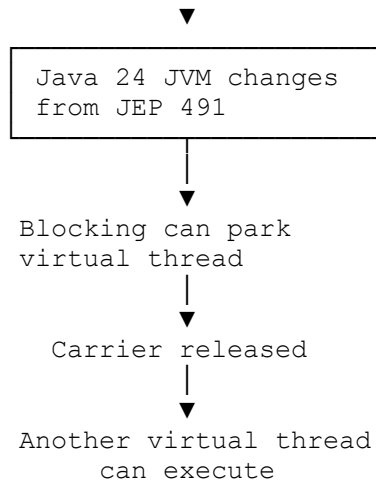
However, ReentrantLock still has capabilities that synchronized does not, such as:

- tryLock()
- Interruptible lock acquisition
- Fairness configuration
- Multiple Condition objects

So it remains useful when those features are required.

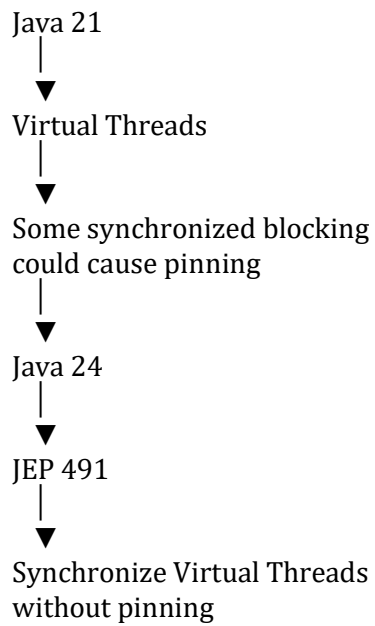
Java 24 Architecture





Relationship With Virtual Threads

The feature is easiest to understand as an improvement to Java 21's virtual-thread model.



Key Takeaways

Property	Details
Java Version	24
Year	2025
JEP	491
Feature	Synchronize Virtual Threads without Pinning
Main area	Virtual Threads / JVM
Java syntax	No change
synchronized	Still supported
Virtual threads	Can synchronize without the previous pinning problem
Carrier threads	Can be freed when virtual threads block
Mutual exclusion	Still preserved
Main benefit	Better scalability for virtual-thread applications
Application migration	Existing synchronization code can generally remain unchanged

One-line definition

Java 24's JEP 491 changes the JVM's implementation of synchronization so that virtual threads can block inside synchronized methods or blocks without pinning their carrier platform threads, improving scalability while preserving normal Java monitor semantics.

Java 24 — Flexible Constructor Bodies (Third Preview)

Java Version: 24

Year: 2025

Feature: Flexible Constructor Bodies (Third Preview)

JEP: 492

UniqueClassName: FlexibleConstructorBodies

Definition

Flexible Constructor Bodies is a **third-preview Java language feature in Java 24** that makes constructors more flexible by allowing statements to appear **before the explicit constructor invocation** (`super(...)` or `this(...)`), subject to important restrictions.

Traditionally, a Java constructor must begin with an explicit or implicit call to another constructor:

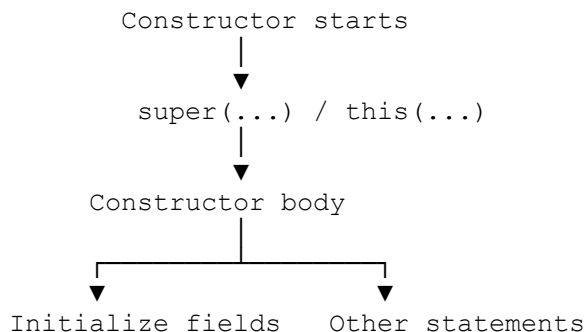
```
class Child extends Parent {  
  
    Child(int value) {  
        super(value); // must come first  
        // other statements  
    }  
}
```

With Flexible Constructor Bodies, Java allows certain **pre-construction statements** before the `super(...)` call.

This is useful when constructor arguments need to be **validated, normalized, calculated, or prepared** before invoking the superclass constructor.

Traditional Constructor

Before this feature, conceptually:



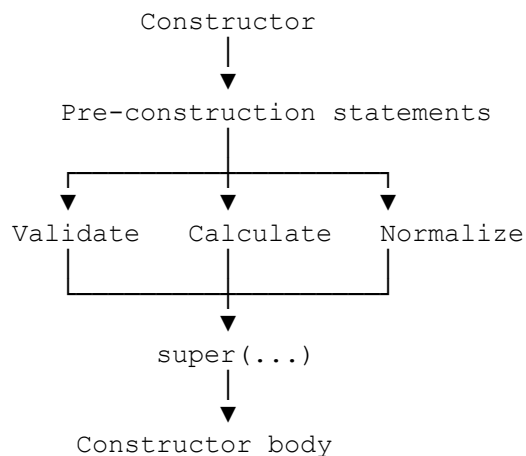
For example:

```
class Person {  
  
    Person(String name) {  
        System.out.println("Creating " + name);  
    }  
}  
  
class Employee extends Person {  
  
    Employee(String name) {  
        super(name);  
        System.out.println("Employee created");  
    }  
}
```

The `super(name)` invocation must occur before ordinary constructor-body statements.

Flexible Constructor Body

Java 24 allows a limited form of:



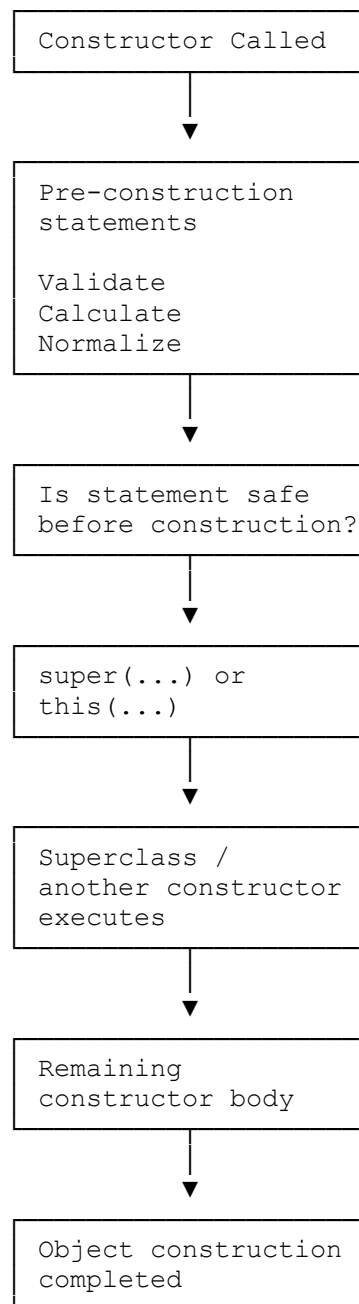
For example:

```
class Employee extends Person {  
  
    Employee(String name) {  
  
        String normalized =  
            name.trim().toUpperCase();
```

```
    super(normalized);  
  }  
}
```

The important new capability is that `normalized` can be computed **before** `super(...)`.

Flowchart



Java Code Example

Here is a simple example demonstrating the idea.

```
public class FlexibleConstructorBodies {

    static class Person {

        private final String name;

        Person(String name) {
            this.name = name;
            System.out.println(
                "Person constructor: " + name
            );
        }
    }

    static class Employee extends Person {

        private final int employeeId;

        Employee(String name, int employeeId) {

            // Pre-construction computation
            String normalizedName =
                name.trim().toUpperCase();

            // Explicit superclass constructor invocation
            super(normalizedName);

            // Normal constructor body
            this.employeeId = employeeId;

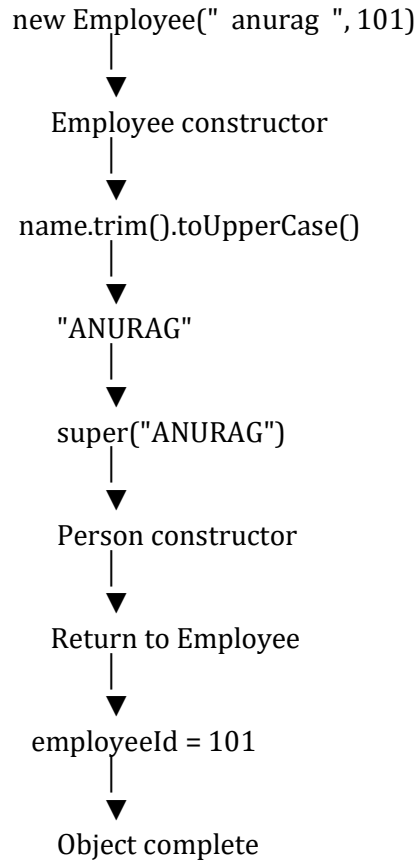
            System.out.println(
                "Employee ID: " + employeeId
            );
        }
    }

    public static void main(String[] args) {

        Employee employee =
            new Employee("  anurag  ", 101);

        System.out.println(
            "Employee created successfully."
        );
    }
}
```

Conceptually, the execution is:



Why Is This Useful?

Consider constructor argument validation.

Traditionally, you might need a static helper method:

```
class Employee extends Person {  
  
    Employee(String name) {  
        super(validateName(name));  
    }  
  
    static String validateName(String name) {  
        if (name == null || name.isBlank()) {  
            throw new IllegalArgumentException(  
                "Name cannot be empty"  
            );  
        }  
    }  
}
```

```

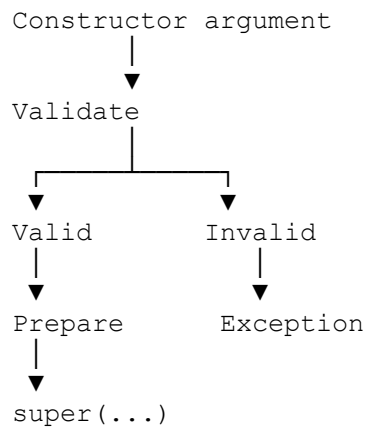
    }

    return name.trim();
}
}

```

Flexible Constructor Bodies allow certain validation or preparation logic to be expressed directly in the constructor before the superclass invocation.

Conceptually:



Example — Validation Before super()

```

public class FlexibleConstructorBodies {

    static class Account {

        Account(String accountNumber) {
            System.out.println(
                "Account: " + accountNumber
            );
        }
    }

    static class BankAccount extends Account {

        BankAccount(String accountNumber) {

            if (accountNumber == null ||
                accountNumber.isBlank()) {

                throw new IllegalArgumentException(
                    "Account number is required"
                );
            }
        }
    }
}

```

```

        String normalized =
            accountNumber.trim();

        super(normalized);

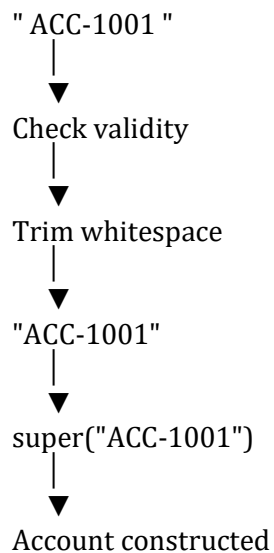
        System.out.println(
            "BankAccount initialized"
        );
    }

    public static void main(String[] args) {

        BankAccount account =
            new BankAccount(" ACC-1001 ");
    }
}

```

The logical sequence becomes:



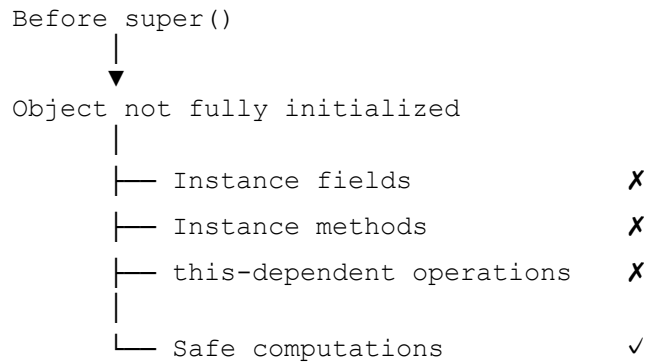
Important Restriction: this Cannot Be Used Freely

The object is **not fully constructed** before the superclass constructor executes.

Therefore, pre-construction statements are subject to restrictions.

For example, you cannot simply access the object's instance state as though construction were already complete.

Conceptually:



The purpose is to prevent code from observing or manipulating an incompletely initialized object.

`super()` vs `this()`

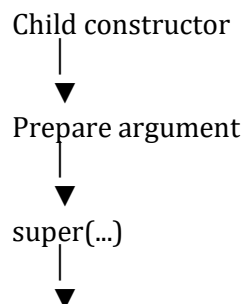
Flexible constructor bodies apply to explicit constructor invocations.

`super(...)`

Calls the superclass constructor:

```
class Child extends Parent {  
    Child(int value) {  
        int adjusted =  
            Math.max(value, 0);  
        super(adjusted);  
    }  
}
```

Flow:



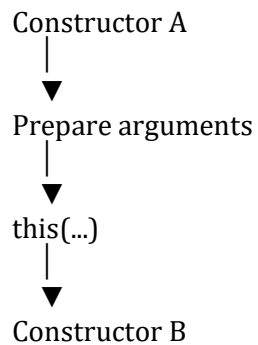
Parent constructor

this(...)

Calls another constructor in the same class:

```
class Person {  
  
    Person(String name) {  
        // ...  
    }  
  
    Person() {  
  
        String defaultName =  
            "Unknown";  
  
        this(defaultName);  
    }  
}
```

Conceptually:



Traditional vs Flexible Constructor

Traditional

```
class Employee extends Person {
```

```
    Employee(String name) {  
        super(name);
```

```
        String normalized =  
            name.trim();
```

```
}  
}
```

The superclass constructor receives the original value.

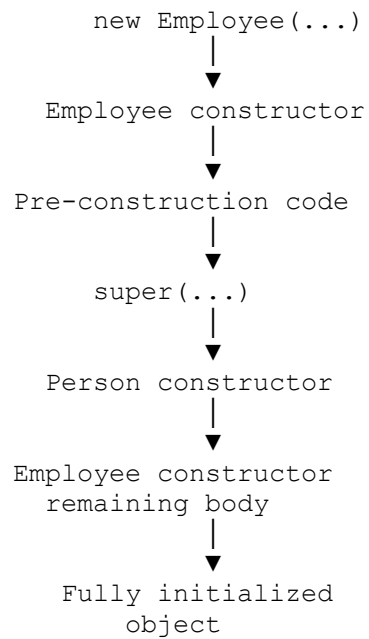
Flexible

```
class Employee extends Person {  
  
    Employee(String name) {  
  
        String normalized =  
            name.trim();  
  
        super(normalized);  
    }  
}
```

Now the value can be prepared before invoking the superclass constructor.

Constructor Execution Model

The overall object-construction process can be viewed as:



Why the Feature Was Introduced

The traditional Java constructor rule can sometimes force developers to move simple preparation logic into:

- Static helper methods
- Factory methods
- Extra constructors
- Temporary variables outside the constructor

Flexible Constructor Bodies make constructors more expressive while preserving the important safety rule that **the superclass must be initialized before the subclass can use its instance state**.

Java 24 Preview Compilation

Because this is a **third-preview feature**, Java 24 requires preview support:

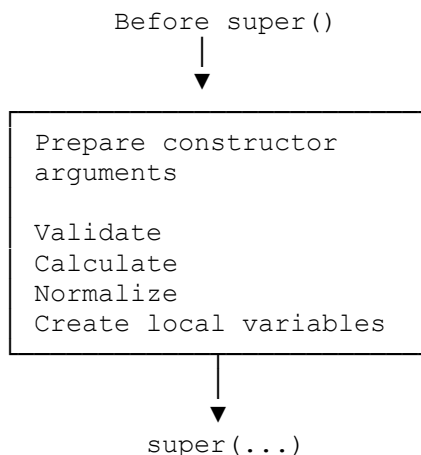
```
javac --enable-preview --release 24 FlexibleConstructorBodies.java
```

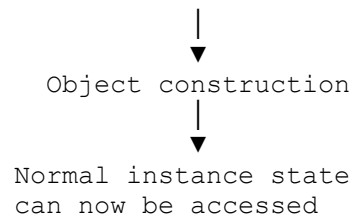
Run:

```
java --enable-preview FlexibleConstructorBodies
```

Important Safety Rule

The key distinction is:





The feature **does not mean arbitrary code can execute before `super()`**. Java still enforces rules that prevent unsafe access to the incompletely constructed object.

Key Takeaways

Property	Details
Java Version	24
Year	2025
JEP	492
Status	Third Preview
Feature	Flexible Constructor Bodies
Main purpose	Allow safe statements before constructor invocation
<code>super(...)</code>	Can follow permitted pre-construction statements
<code>this(...)</code>	Also part of the flexible constructor model
Useful for	Validation, normalization, argument preparation
Object state	Cannot be accessed arbitrarily before superclass construction
Java syntax	Extended constructor syntax
Compilation	Requires <code>--enable-preview</code>

One-line definition

Java 24's Flexible Constructor Bodies (JEP 492) allows constructors to perform permitted pre-construction computations and validations before invoking `super(...)` or `this(...)`, making constructor code more expressive while preserving Java's object-initialization safety rules.

Java 24 — Linking Run-Time Images without JMODs

Java Version: 24

Year: 2025

Feature: Linking Run-Time Images without JMODs

JEP: 493

UniqueClassName: LinkRuntimeImages

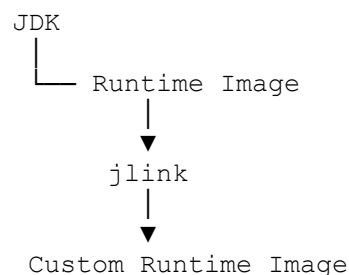
Definition

Linking Run-Time Images without JMODs is a **Java 24 feature** that enhances the `jlink` tool so that it can create a custom Java **run-time image directly from a JDK installation**, without requiring the JDK's internal **JMOD files** as input.

Traditionally, `jlink` builds a custom runtime image from JMOD files:



With Java 24, `jlink` can use the **JDK modules directly from the JDK's runtime image**, making it possible to create a custom runtime image without shipping or retaining the JMOD files.

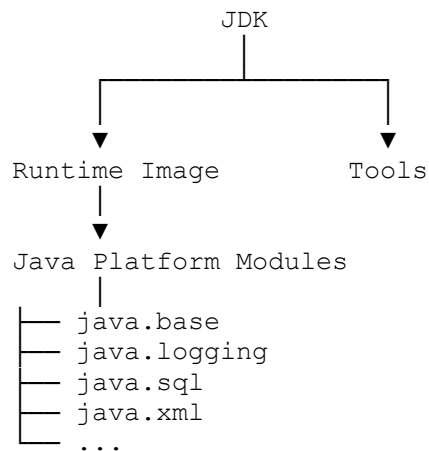


This is particularly useful for **JDK distributions that do not contain JMOD files**, reducing the size and distribution requirements of a JDK.

What Is a Runtime Image?

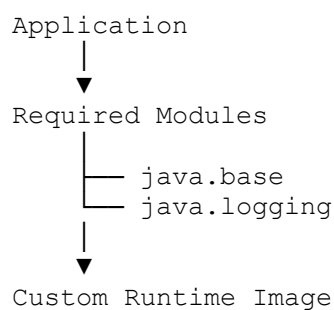
A normal JDK contains a runtime image holding Java platform classes and resources.

Conceptually:



A **custom runtime image** contains only the modules needed by an application.

For example:

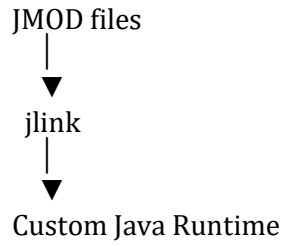


This can make application distributions smaller than shipping a complete JDK.

What Is jlink?

jlink is a JDK command-line tool used to assemble a custom Java runtime image.

Traditional model:



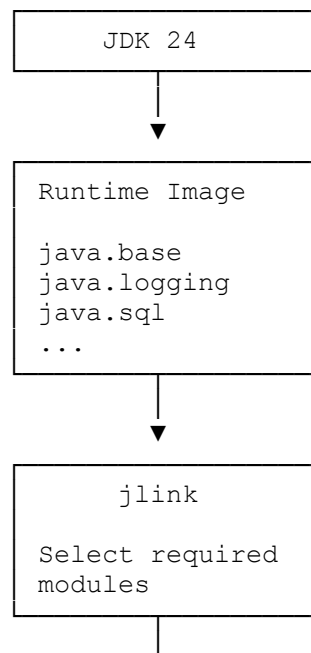
For example:

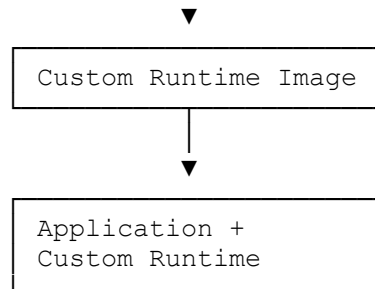
```
jlink --add-modules java.base,java.logging \
      --output custom-runtime
```

The result is a runtime image:

```
custom-runtime/
├── bin/
│   └── java
│       ...
├── conf/
├── lib/
└── ...
```

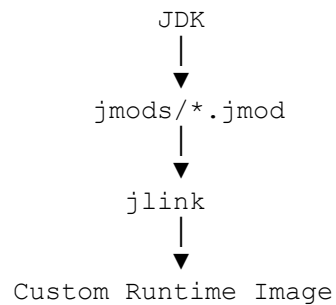
Flowchart





Before Java 24

The traditional jlink implementation relied on JMOD files:

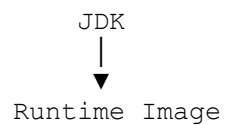


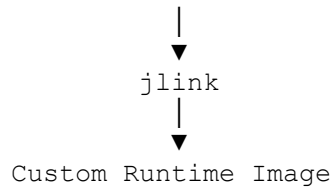
For example:

```
JDK/
├─ jmods/
│   ├── java.base.jmod
│   ├── java.compiler.jmod
│   ├── java.desktop.jmod
│   ├── java.logging.jmod
│   ├── java.sql.jmod
│   └─ ...
```

Java 24

JDK 24 allows the required modules to be obtained from the JDK's **runtime image** instead:





The JMOD files no longer have to be present for this use case.

Java Code Example

This feature is primarily a **JDK tooling feature**, not a Java language feature.

Therefore, there is no special Java syntax.

We can create a normal modular Java application.

module-info.java

```
module example.app {
    requires java.logging;
}
```

LinkRuntimeImages.java

```
import java.util.logging.Logger;

public class LinkRuntimeImages {

    private static final Logger LOGGER =
        Logger.getLogger(
            LinkRuntimeImages.class.getName()
        );

    public static void main(String[] args) {

        LOGGER.info(
            "Running from a custom Java runtime image."
        );

        System.out.println(
            "Java Version: "
            + System.getProperty("java.version")
        );

    }
}
```

Project Structure

```
LinkRuntimeImages/
├── src/
│   ├── example.app/
│   │   ├── module-info.java
│   │   └── LinkRuntimeImages.java
│   └── runtime/
```

Compile:

```
javac -d out \
    src/example.app/module-info.java \
    src/example.app/LinkRuntimeImages.java
```

Create a JAR:

```
jar --create \
    --file app.jar \
    --main-class LinkRuntimeImages \
    -C out .
```

Then jlink can create a custom runtime containing the required modules.

Creating the Runtime Image

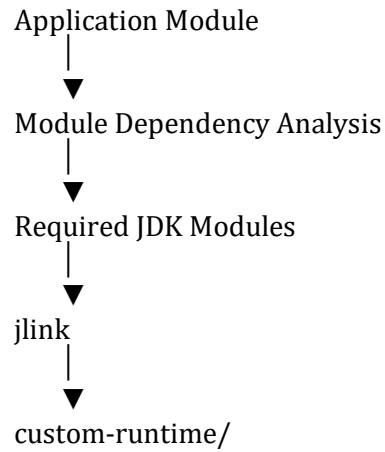
For a modular application, the typical command is:

```
jlink --module-path "$JAVA_HOME/jmods:app.jar" \
    --add-modules example.app \
    --output custom-runtime
```

On Windows, the module-path separator is different:

```
jlink --module-path "%JAVA_HOME%\jmods;app.jar" ^
    --add-modules example.app ^
    --output custom-runtime
```

The important concept is:



The exact command-line form can vary depending on how the application modules and JDK runtime image are supplied.

Running the Custom Runtime

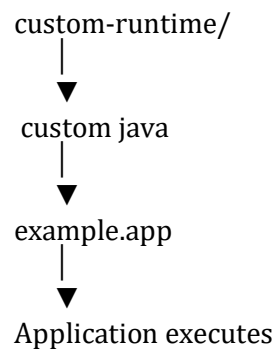
After jlink creates the image:

```
custom-runtime/bin/java \  
-m example.app
```

On Windows:

```
custom-runtime\bin\java.exe -m example.app
```

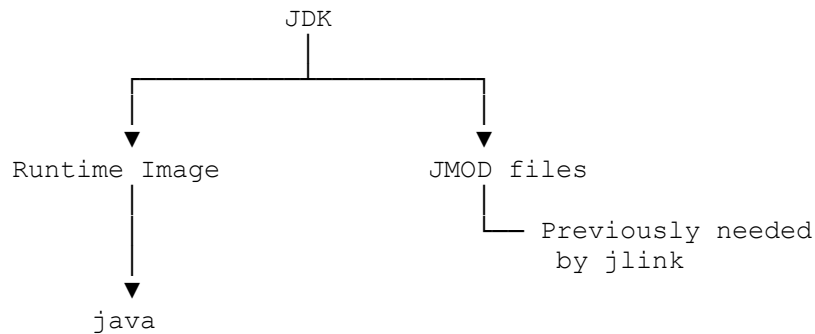
Conceptually:



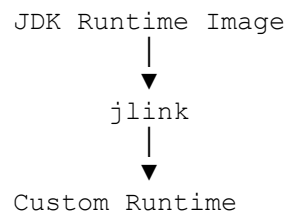
Why Removing the JMOD Requirement Matters

JMOD files are useful during JDK construction and tooling, but they are not required by the Java runtime itself.

A JDK distribution can therefore be thought of as:



With JEP 493, jlink can work from the runtime image.



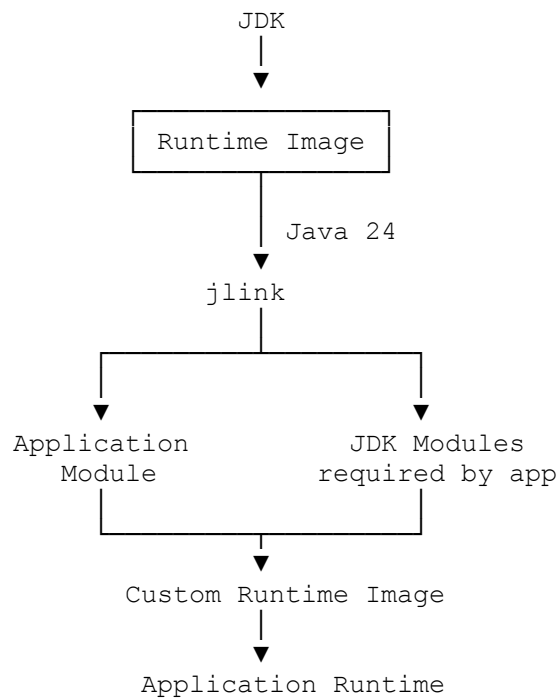
This can enable **smaller JDK distributions** because a vendor or distribution does not necessarily need to include JMOD files solely to support jlink.

Traditional vs Java 24

Aspect	Traditional jlink	Java 24
JDK runtime image	Used	Used
JMOD files	Required for traditional linking	Not required
Custom runtime images	✓	✓
jlink	✓	✓

Java source syntax	No change	No change
Main benefit	Custom runtimes	Custom runtimes from JDK runtime image
JDK distribution	Often includes JMODs for linking	Can omit JMODs where appropriate

Runtime Image Architecture



Deployment Example

This feature is particularly useful when packaging an application for deployment.

Instead of:

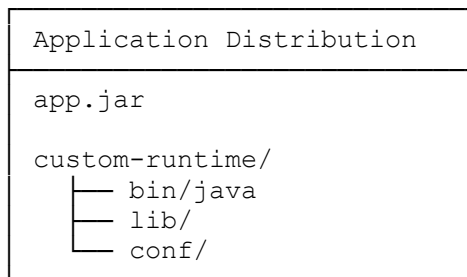
Application
+
Complete JDK

↓
Large deployment

you can create:

Application
+
Custom Runtime Image
↓
Smaller deployment

For example:



The application can then run without requiring a separately installed full JDK.

Important Distinction

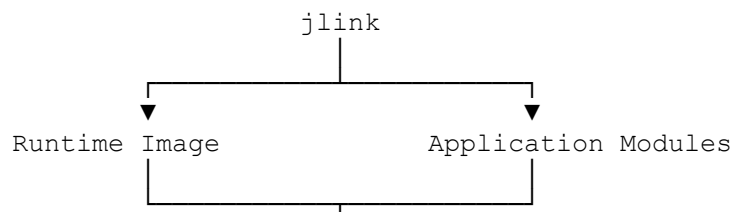
This feature does **not** mean:

"jlink is removed."

It means:

"jlink no longer fundamentally depends on JMOD files being present in the JDK distribution."

The relationship becomes:



Key Takeaways

Property	Details
Java Version	24
Year	2025
JEP	493
Feature	Linking Run-Time Images without JMODs
Tool	Jlink
Primary area	JDK tooling / runtime images
JMOD files	No longer required for linking
Runtime image	Used directly as a source
Java syntax changes	None
Main benefit	Enables leaner JDK distributions and custom runtime creation
Application changes	Generally none
Deployment benefit	Potentially smaller JDK/distribution footprint

One-line definition

Java 24's JEP 493 enables jlink to create custom run-time images directly from a JDK's run-time image without requiring JMOD files, making it possible to distribute leaner JDKs while retaining the ability to build application-specific Java runtimes.

Java 24 — Module Import Declarations (Second Preview)

Java Version: 24

Year: 2025

Feature: Module Import Declarations (Second Preview)

JEP: 494

UniqueClassName: ModuleImportDeclarations

Definition

Module Import Declarations is a **second-preview Java language feature in Java 24** that allows a Java source file to import **all the accessible types exported by the modules it specifies**, rather than requiring developers to write many individual imports or wildcard package imports.

The new syntax is:

```
import module java.sql;
```

Instead of writing:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
```

you can use:

```
import module java.sql;
```

The compiler determines the packages exported by the module and makes their accessible types available for simple-name use.

Traditional Import Model

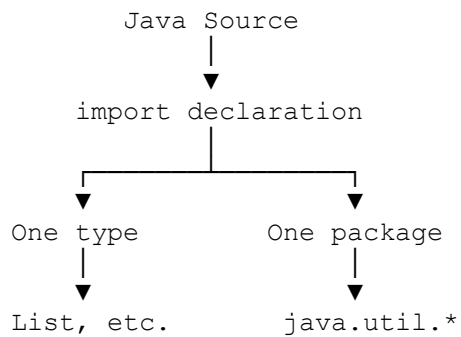
Java normally imports types by package:

```
import java.util.List;
import java.util.ArrayList;
import java.util.HashMap;
```

Or all types from one package:

```
import java.util.*;
```

The relationship is:



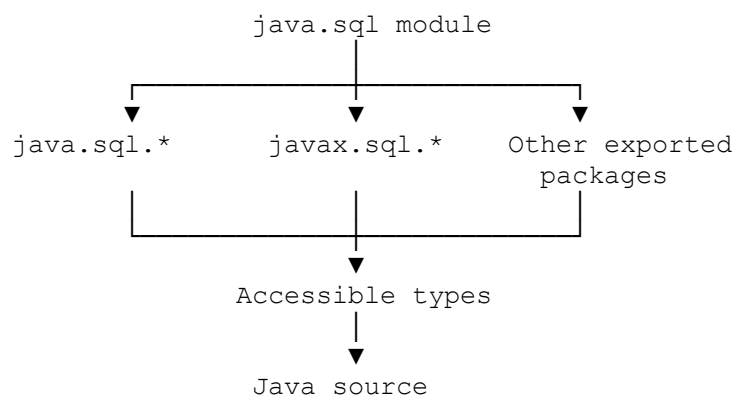
The limitation is that a package wildcard only covers **one package**.

Module Import Model

With Java 24:

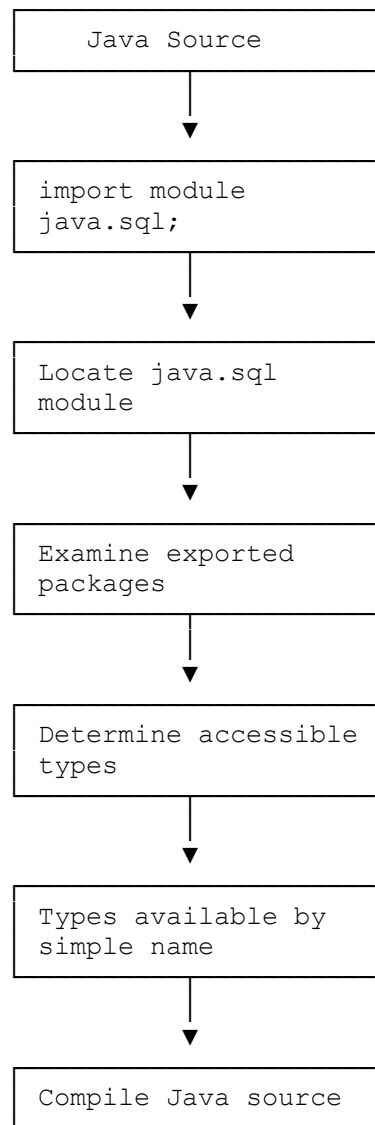
```
import module java.sql;
```

the compiler looks at the module:



This provides a module-level way of importing types.

Flowchart



Java Code Example

Consider a simple JDBC program.

Without module import declarations, you might write:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
```

```
import java.sql.ResultSet;
import java.sql.SQLException;
```

With a module import:

```
import module java.sql;

public class ModuleImportDeclarations {

    public static void main(String[] args)
        throws SQLException {

        String url =
            "jdbc:h2:mem:testdb";

        try (Connection connection =
            DriverManager.getConnection(url)) {

            System.out.println(
                "Database connection created."
            );

        }

    }
}
```

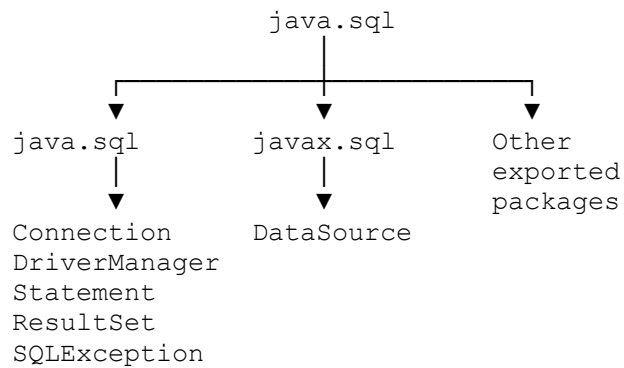
The important line is:

```
import module java.sql;
```

It gives the source file access to accessible types exported by the java.sql module.

What Does java.sql Contain?

Conceptually:



Therefore:

```
import module java.sql;
```

can make types such as:

```
Connection  
DriverManager  
Statement  
ResultSet  
SQLException  
DataSource
```

available by simple name, subject to the normal accessibility and import-resolution rules.

Comparison

Traditional Individual Imports

```
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.ResultSet;  
import java.sql.SQLException;  
import java.sql.Statement;
```

Package Wildcard

```
import java.sql.*;
```

Module Import

```
import module java.sql;
```

Conceptually:

```
Individual import  
└─ One specific type  
  
Package wildcard  
└─ All accessible types in one package  
  
Module import  
└─ Accessible types from  
    exported packages of a module
```

Why Is This Different From `java.sql.*`?

This is an important distinction.

```
import java.sql.*;
```

means:

Import accessible types from the `java.sql` package.

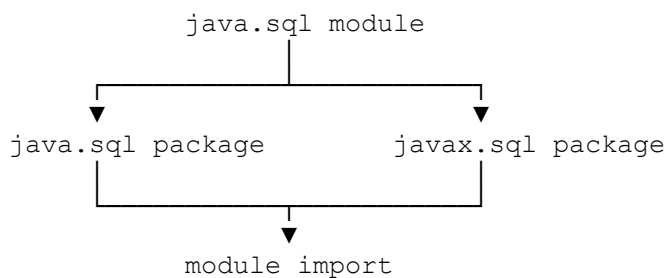
Whereas:

```
import module java.sql;
```

means:

Import accessible types from the packages exported by the `java.sql` module.

So:



This becomes particularly useful when a module exports multiple packages.

Example With Multiple Types

```
import module java.base;
import module java.sql;

public class ModuleImportDeclarations {

    public static void main(String[] args)
        throws Exception {
```

```

String url =
    "jdbc:h2:mem:testdb";

System.out.println(
    "Starting database example..."
);

try (Connection connection =
    DriverManager.getConnection(url)) {

    System.out.println(
        "Connected: "
        + connection.isValid(1)
    );
}
}

```

Here:

```
import module java.base;
```

provides types from the `java.base` module, while:

```
import module java.sql;
```

provides JDBC-related types.

Note: `java.base` is implicitly available to every Java source file, so explicitly importing it is generally unnecessary.

Module Import and `module-info.java`

Module import declarations are related to the **Java Platform Module System (JPMS)**.

For example:

```

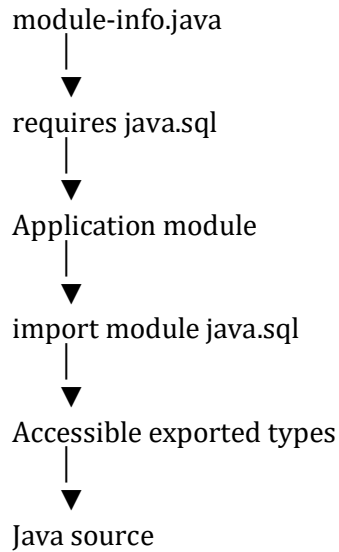
module my.application {
    requires java.sql;
}

```

Then a source file can use:

```
import module java.sql;
```

Conceptually:



The requires directive and import module declaration serve **different purposes**.

requires

Controls **module readability/dependency**:

requires java.sql;

import module

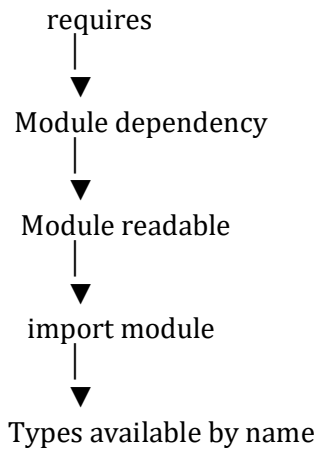
Controls **which accessible types are available by simple name**:

import module java.sql;

requires vs import module

Declaration	Purpose
requires java.sql;	Establishes module dependency/readability
import module java.sql;	Imports accessible types from the module's exported packages
import java.sql.*;	Imports accessible types from one package

Therefore:



Ambiguous Type Names

Because a module can expose types from multiple packages, module imports can potentially introduce **name conflicts**.

For example, imagine two modules export types with the same simple name:

```
Module A
└─ package.a.Type

Module B
└─ package.b.Type
```

If both are imported:

```
import module moduleA;
import module moduleB;
```

then:

Type value;

could be ambiguous.

The compiler's normal import-resolution rules apply, and explicit imports can be used to resolve such conflicts.

Conceptually:

```
import module A
    |
    └─ Type

import module B
    |
    └─ Type
        |
        ▼
    Ambiguous?
        |
        ▼
    Use explicit import
```

Why Was This Feature Introduced?

The feature is especially useful for **large modules** and for simplifying source code that uses many APIs from the same module.

Traditional:

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
```

With module import:

```
import module java.net.http;
```

The source becomes more concise.

Example — HTTP Client

```
import module java.net.http;

import java.net.URI;

public class ModuleImportDeclarations {

    public static void main(String[] args)
        throws Exception {
```

```

HttpClient client =
    HttpClient.newHttpClient();

HttpRequest request =
    HttpRequest.newBuilder()
        .uri(
            URI.create(
                "https://example.com"
            )
        )
        .GET()
        .build();

HttpResponse<String> response =
    client.send(
        request,
        HttpResponse.BodyHandlers.ofString()
    );

System.out.println(
    "Status: "
    + response.statusCode()
);
}
}

```

Here:

`import module java.net.http;`

makes the HTTP client API types available without listing each one individually.

Source-Code Simplification

Before

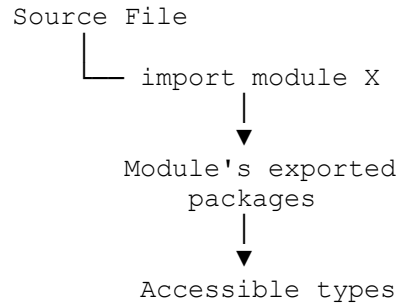
Source File

```

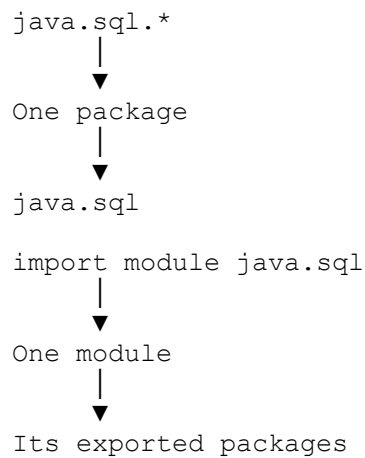
|— import Type1
|— import Type2
|— import Type3
|— import Type4
|— import Type5
|— ...

```

With Module Import



Module Import vs Wildcard Import



This is the central idea behind the feature.

Java 24 Preview Compilation

Because **Module Import Declarations** is a **second-preview feature in Java 24**, preview support is required.

Compile:

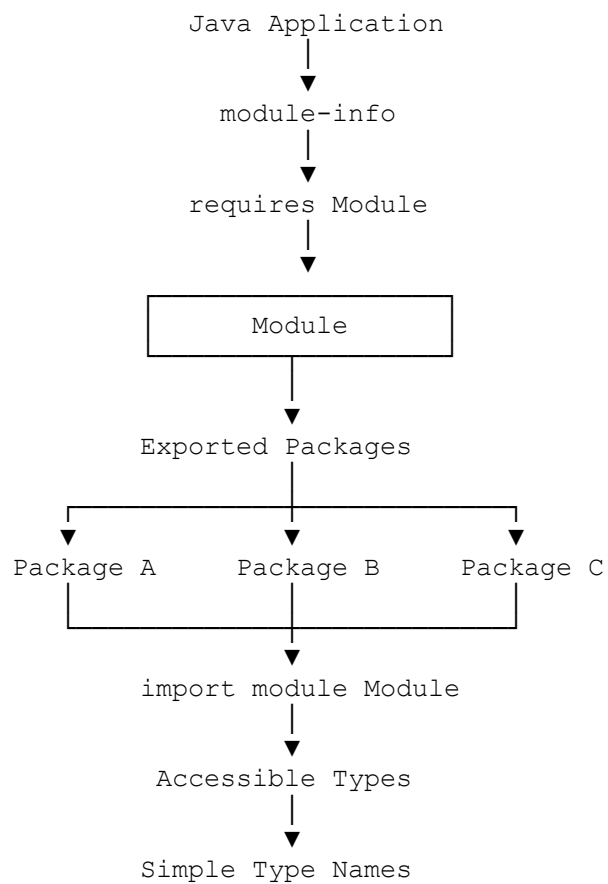
```
javac --enable-preview --release 24 ModuleImportDeclarations.java
```

Run:

```
java --enable-preview ModuleImportDeclarations
```

For a modular application, you would also compile and run with the appropriate module-path/module options.

Overall Architecture



Key Takeaways

Property	Details
Java Version	24
Year	2025
JEP	494
Status	Second Preview
Feature	Module Import Declarations

Syntax	import module module.name;
Purpose	Import accessible types from a module's exported packages
Scope	Module-level
Different from package.*	Yes
Related technology	JPMS
Java syntax change	Yes
Compilation	Requires --enable-preview

One-line definition

Java 24's Module Import Declarations (JEP 494) allow a source file to import accessible types from the exported packages of an entire module using import module module.name;, providing a concise alternative to numerous individual or package-level import declarations.

Java 24 — Simple Source Files and Instance Main Methods (Fourth Preview)

Java Version: 24

Year: 2025

Feature: Simple Source Files and Instance Main Methods (Fourth Preview)

JEP: 495

UniqueClassName: SimpleSourceFilesInstanceMain

Definition

Simple Source Files and Instance Main Methods is a **fourth-preview Java language feature in Java 24** that makes it easier for beginners to write and run small Java programs without first learning the full ceremony of classes, public static void main(String[] args), and explicit public modifiers.

The goal is to let a simple Java source file look more like a straightforward program:

```
void main() {
    println("Hello, World!");
}
```

Instead of the traditional:

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

The feature is intended to provide a **gentler entry point into Java**, while remaining compatible with Java's existing object-oriented model.

Traditional Java Program

A beginner traditionally starts with:

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
  
        System.out.println(  
            "Hello, World!"  
        );  
    }  
}
```

There is a lot of syntax to understand before the student reaches the actual logic:

```
public class HelloWorld  
    |  
    ▼  
static void main(String[] args)  
    |  
    ▼  
System.out.println(...)  
    |  
    ▼  
Hello, World!
```

For a first programming lesson, much of this syntax is incidental.

Simple Source File

With the preview feature, a simple source file can contain:

```
void main() {  
    IO.println("Hello, World!");  
}
```

The basic structure becomes:

Simple Source File



void main()



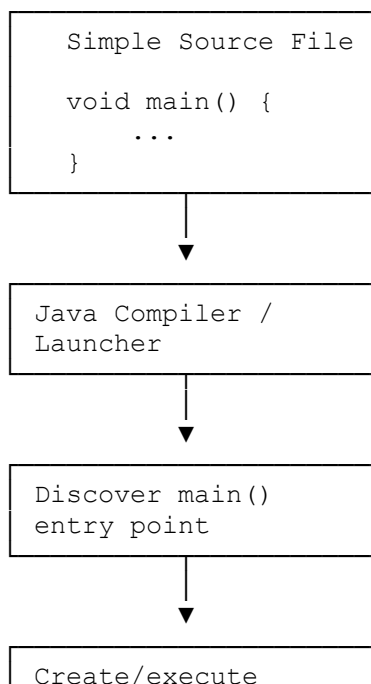
Program

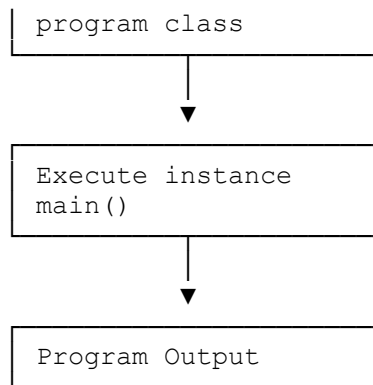


Output

The compiler/runtime supplies the necessary surrounding machinery.

Flowchart





Java Code Example 1 — Simplest Program

A simple Java 24 source file can be:

```
void main() {  
    IO.println("Hello, World!");  
}
```

The source file does not explicitly declare:

```
class HelloWorld
```

or:

```
public static void main(String[] args)
```

The focus is directly on the program logic.

Java Code Example 2 — Variables and Logic

You can write ordinary Java statements:

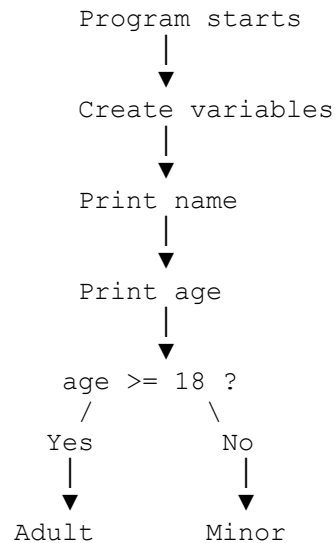
```
void main() {  
  
    String name = "Anurag";  
    int age = 25;  
  
    IO.println("Name: " + name);  
    IO.println("Age: " + age);  
  
    if (age >= 18) {
```

```

        IO.println("Adult");
    } else {
        IO.println("Minor");
    }
}

```

Conceptually:



Java Code Example 3 — Instance Main Method

The feature also allows an **instance main() method**.

For example:

```

class SimpleSourceFilesInstanceMain {

    void main() {

        IO.println(
            "Hello from an instance main method!"
        );
    }
}

```

Notice:

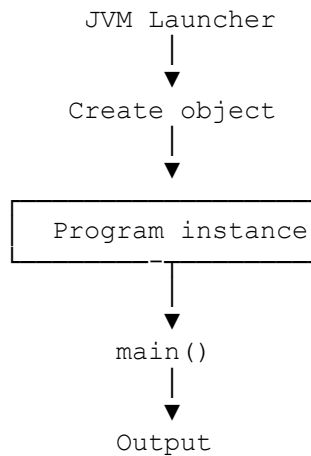
void main()

is **not**:

```
public static void main(String[] args)
```

It is an **instance method**.

Conceptually:



Why Instance main()?

Traditional Java requires:

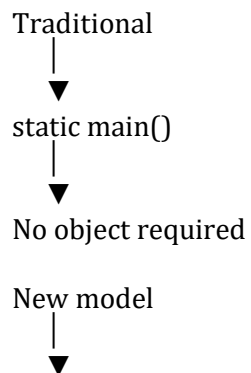
```
public static void main(String[] args)
```

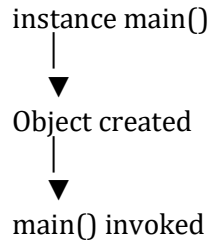
The static keyword means the JVM can invoke main() without first creating an instance of the class.

The preview feature allows an instance entry point:

```
void main()
```

The runtime can create an instance and invoke it.





This allows introductory Java programs to teach **objects and instance methods** naturally.

Simple Source File + Instance Main

A compact example:

```
void main() {  
    var message =  
        "Java 24 is simpler!";  
    IO.println(message);  
}
```

The conceptual class generated/used by the Java implementation contains an instance `main()` method.

You can think of it approximately as:

```
class GeneratedProgram {  
    void main() {  
        var message =  
            "Java 24 is simpler!";  
        IO.println(message);  
    }  
}
```

The important point is that the programmer does **not need to write this wrapper** in the simple-source-file case.

IO.println()

The example uses:

```
IO.println(...)
```

The IO class is part of the preview feature's simplified programming model.

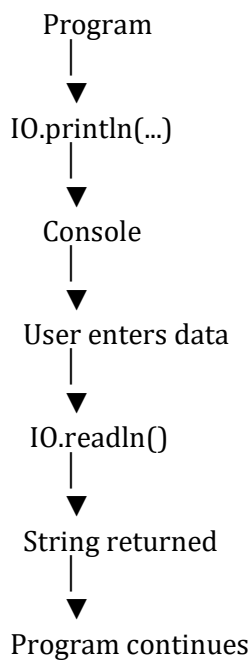
It is designed to make basic input/output easier to discover than:

```
System.out.println(...)
```

For example:

```
void main() {  
    IO.println("Enter your name:");  
    String name = IO.readLine();  
    IO.println(  
        "Hello, " + name  
    );  
}
```

Conceptually:



Example — Interactive Program

```
void main() {  
    IO.println("What is your name?");  
    String name = IO.readLine();  
    IO.println(  
        "Welcome, " + name + "!"  
    );  
}
```

Possible execution:

```
What is your name?  
Anurag  
Welcome, Anurag!
```

This style is especially convenient for introductory programming exercises.

Traditional vs Simple Source

Traditional Java

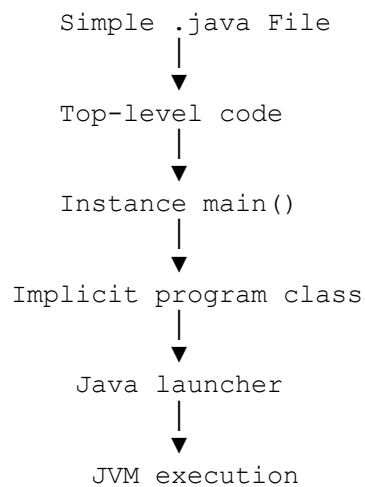
```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println(  
            "Hello, World!"  
        );  
    }  
}
```

Simple Source File

```
void main() {  
    IO.println(  
        "Hello, World!"  
    );  
}
```

The amount of boilerplate is significantly reduced.

Source File Architecture



Important: This Is Still Java

The feature does **not** introduce a new programming language.

You can still use:

String
int
if
else
for
while
record
class
interface
enum

and normal Java APIs.

For example:

```
void main() {
```

```
var numbers =  
    java.util.List.of(  
        10, 20, 30, 40  
    );  
  
for (var number : numbers) {  
    IO.println(number);  
}  
}
```

Output:

```
10  
20  
30  
40
```

Can You Still Write Classes?

Yes.

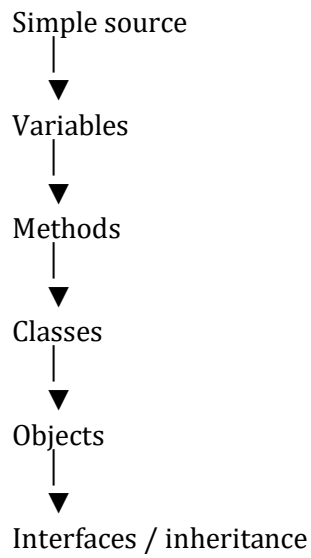
The feature is designed to coexist with ordinary Java classes.

For example:

```
class Calculator {  
  
    int add(int a, int b) {  
        return a + b;  
    }  
}  
  
void main() {  
  
    Calculator calculator =  
        new Calculator();  
  
    int result =  
        calculator.add(10, 20);  
  
    IO.println(result);  
}
```

Output:

The program can therefore introduce object-oriented concepts progressively.

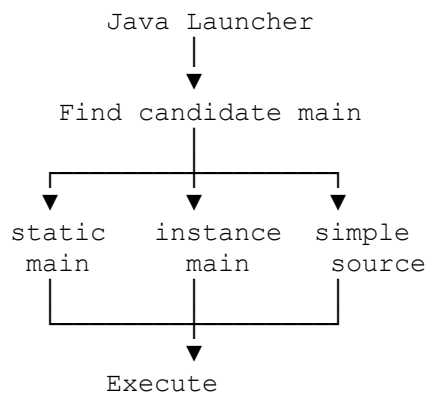


Comparison of Main Methods

Main form	Traditional	Java 24 preview
public static void main(String[] args)	✓	✓
static void main()	—	✓
void main()	—	✓
Instance main	✗	✓
Simple source file	✗	✓
Explicit class required	Usually	Not necessarily
IO.println()	—	✓

How the Launcher Thinks About It

Conceptually, the launcher looks for an appropriate main entry point:



The exact launch-selection rules are defined by the JEP and are more precise than this simplified diagram.

Compilation

Because this is a **preview feature in Java 24**, preview support is required.

Compile:

```
javac --enable-preview --release 24 SimpleSourceFilesInstanceMain.java
```

Run:

```
java --enable-preview SimpleSourceFilesInstanceMain
```

For a source-file launcher workflow, Java's source-file launching mechanism can also be used with preview enabled as appropriate.

Why Is This Important for Beginners?

Traditional Java immediately exposes beginners to several concepts:

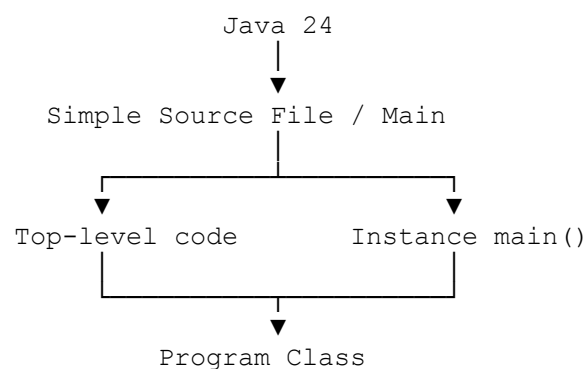
```
public
class
static
void
main
String[]
System.out
```

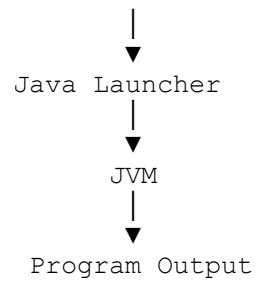
The new model allows the first lesson to focus on:

```
program
|
▼
statement
|
▼
variable
|
▼
condition
|
▼
loop
|
▼
method
|
▼
object
```

So Java can introduce programming concepts incrementally rather than requiring all the traditional boilerplate at the beginning.

Overall Architecture





Key Takeaways

Property	Details
Java Version	24
Year	2025
JEP	495
Status	Fourth Preview
Feature	Simple Source Files and Instance Main Methods
UniqueClassName	SimpleSourceFilesInstanceMain
Primary goal	Make Java easier to learn and write
Explicit class required	Not for simple source files
Instance main()	Supported
Traditional main()	Still supported
Simplified I/O	IO.println(), IO.readln()
Java syntax	Simplified entry-point/source-file model
Compilation	Requires --enable-preview

One-line definition

Java 24's Simple Source Files and Instance Main Methods (JEP 495) is a fourth-preview feature that simplifies Java programs by allowing top-level program code and instance main() methods, reducing the boilerplate required for small programs while retaining the full Java language and object-oriented programming model.

Java 24 — Quantum-Resistant ML-KEM

Java Version: 24

Year: 2025

Feature: Quantum-Resistant ML-KEM

JEP: 496

UniqueClassName: QuantumResistantMLKEM

Definition

Quantum-Resistant ML-KEM adds support for **ML-KEM (Module-Lattice-Based Key-Encapsulation Mechanism)** to Java's standard cryptographic APIs.

ML-KEM is a **post-quantum cryptographic (PQC)** algorithm designed to establish a shared secret between two parties while being resistant to attacks from both **classical computers** and sufficiently powerful **quantum computers**.

Java 24 standardizes ML-KEM through the Java Cryptography Architecture (JCA).

The primary API is:

`javax.crypto.KEM`

with ML-KEM algorithm support through the JCA/JCE providers.

What Problem Does ML-KEM Solve?

Traditional public-key algorithms such as:

RSA

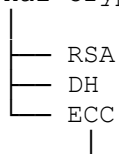
Elliptic-Curve Cryptography (ECC)

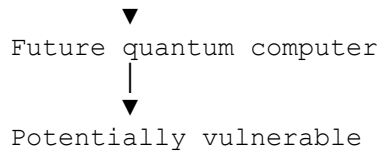
Diffie-Hellman

are vulnerable to sufficiently powerful quantum computers.

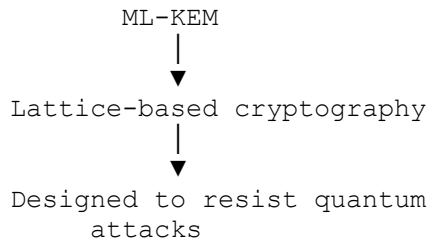
Conceptually:

Traditional Cryptography





ML-KEM belongs to the post-quantum family:

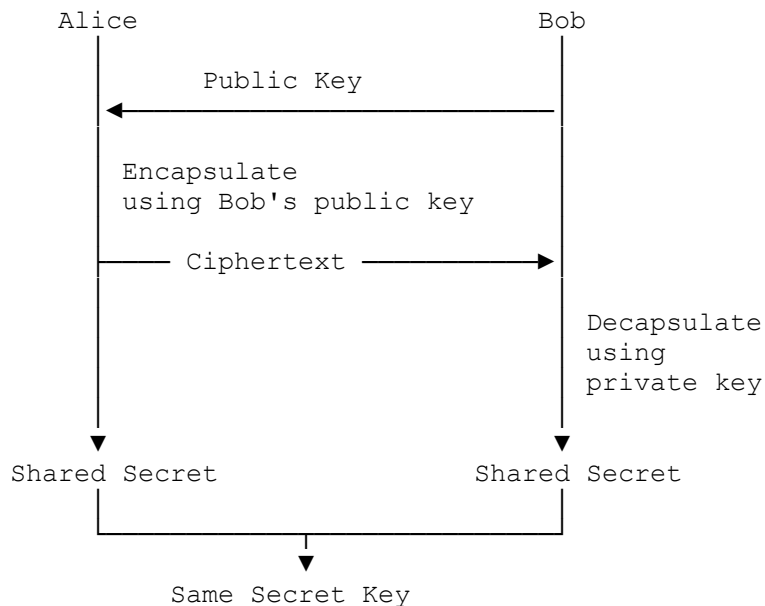


What Is KEM?

KEM means **Key Encapsulation Mechanism**.

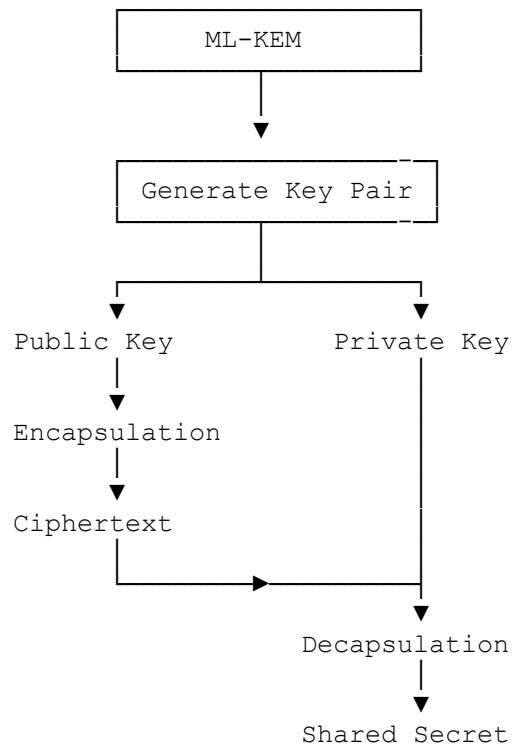
A KEM allows two parties to establish a shared secret using a public/private key pair.

The basic process is:



The secret can then be used with a symmetric encryption algorithm such as AES.

ML-KEM Flowchart



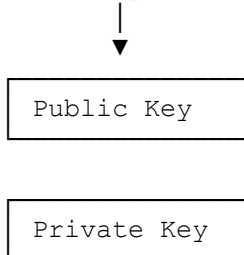
Three Main ML-KEM Operations

ML-KEM provides three fundamental operations.

1. Key Generation

Generate:

ML-KEM Key Generation

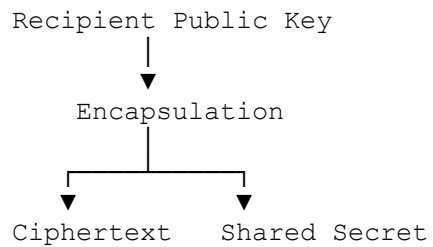


The public key can be shared.

The private key must remain secret.

2. Encapsulation

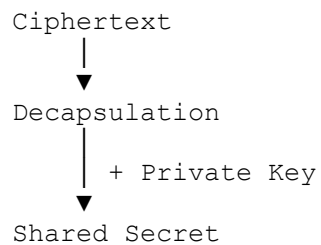
The sender uses the recipient's public key:



The ciphertext is sent to the recipient.

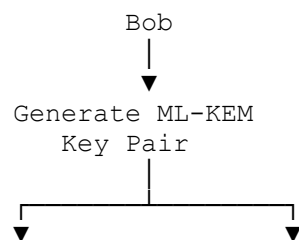
3. Decapsulation

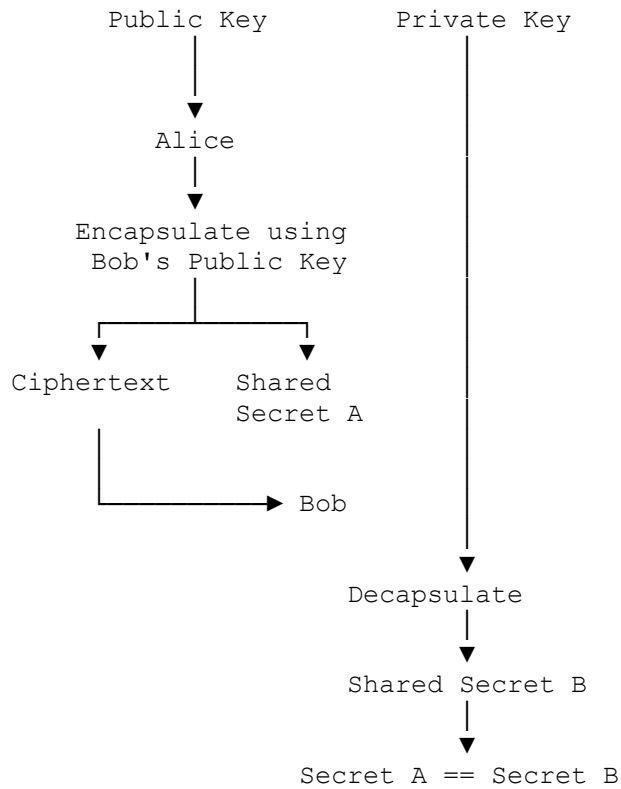
The recipient uses the private key:



Both parties obtain the same shared secret.

Complete Flowchart





Java Code Example

Java 24 provides a standard KEM API.

A basic ML-KEM example:

```
import javax.crypto.KEM;
import javax.crypto.SecretKey;
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.PrivateKey;
import java.security.PublicKey;
import java.security.SecureRandom;

public class QuantumResistantMLKEM {

    public static void main(String[] args)
        throws Exception {

        // Generate ML-KEM key pair
        KeyPairGenerator keyPairGenerator =
            KeyPairGenerator.getInstance("ML-KEM");

        keyPairGenerator.initialize(
            768,
```

```

        new SecureRandom()
    );

    KeyPair keyPair =
        keyPairGenerator.generateKeyPair();

    PublicKey publicKey =
        keyPair.getPublic();

    PrivateKey privateKey =
        keyPair.getPrivate();

    System.out.println(
        "ML-KEM key pair generated."
    );

    // Encapsulation
    KEM kem =
        KEM.getInstance("ML-KEM");

    KEM.Encapsulator encapsulator =
        kem.newEncapsulator(publicKey);

    KEM.Encapsulated encapsulated =
        encapsulator.encapsulate();

    SecretKey senderSecret =
        encapsulated.key();

    byte[] ciphertext =
        encapsulated.encapsulation();

    System.out.println(
        "Ciphertext size: "
        + ciphertext.length
    );

    // Decapsulation
    KEM.Decapsulator decapsulator =
        kem.newDecapsulator(privateKey);

    SecretKey receiverSecret =
        decapsulator.decapsulate(
            ciphertext
        );

    System.out.println(
        "Shared secrets equal: "
        + senderSecret.equals(receiverSecret)
    );
}
}

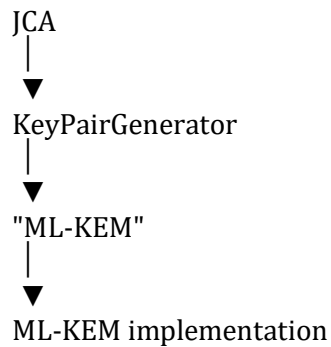
```

Understanding the Code

1. Create the Key Pair Generator

```
KeyPairGenerator keyPairGenerator =  
    KeyPairGenerator.getInstance("ML-KEM");
```

This asks the Java cryptographic framework for an ML-KEM key-pair generator.



2. Select ML-KEM Parameter Set

The example uses:

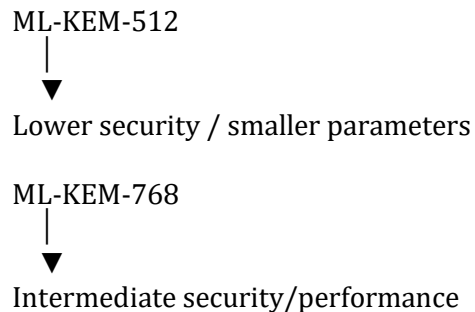
```
keyPairGenerator.initialize(768);
```

ML-KEM has standardized parameter sets:

ML-KEM-512
ML-KEM-768
ML-KEM-1024

They provide different security/performance trade-offs.

Conceptually:



ML-KEM-1024



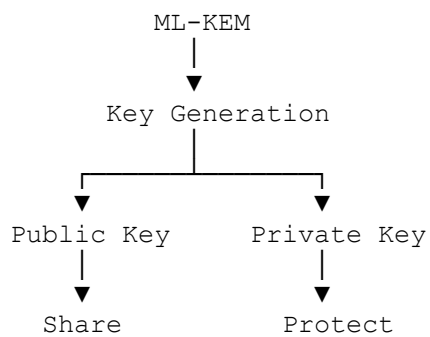
Higher security / larger parameters

For new production cryptographic designs, parameter selection should follow current security guidance and the protocol/library requirements rather than simply choosing the largest parameter set.

3. Generate Keys

```
KeyPair keyPair =  
    keyPairGenerator.generateKeyPair();
```

Produces:



4. Create KEM

```
KEM kem =  
    KEM.getInstance("ML-KEM");
```

KEM is the standard Java API abstraction for key encapsulation.

Application



javax.crypto.KEM



ML-KEM Provider



Cryptographic implementation

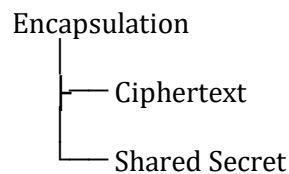
5. Encapsulate

```
KEM.Encapsulator encapsulator =  
    kem.newEncapsulator(publicKey);
```

The sender can then create:

```
KEM.Encapsulated encapsulated =  
    encapsulator.encapsulate();
```

This produces two important values:



The ciphertext is transmitted to the recipient.

The shared secret remains secret.

6. Decapsulate

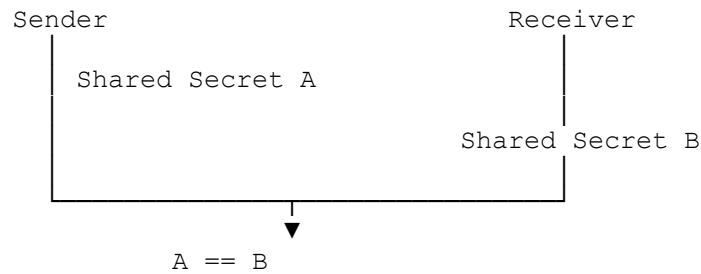
The recipient uses the private key:

```
KEM.Decapsulator decapsulator =  
    kem.newDecapsulator(privateKey);
```

Then:

```
SecretKey receiverSecret =  
    decapsulator.decapsulate(  
        ciphertext  
    );
```

The receiver obtains the same shared secret.



Example Output

The program will produce output similar to:

```
ML-KEM key pair generated.  
Ciphertext size: ...  
Shared secrets equal: true
```

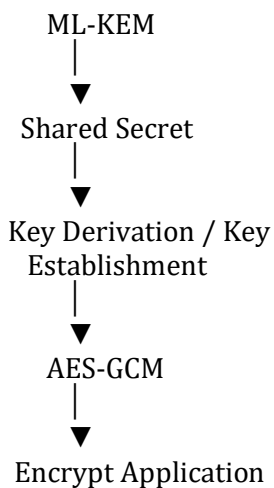
The exact ciphertext size depends on the ML-KEM parameter set and provider implementation.

ML-KEM + Symmetric Encryption

ML-KEM itself is **not normally used to encrypt large application messages directly**.

Instead, it establishes a shared secret.

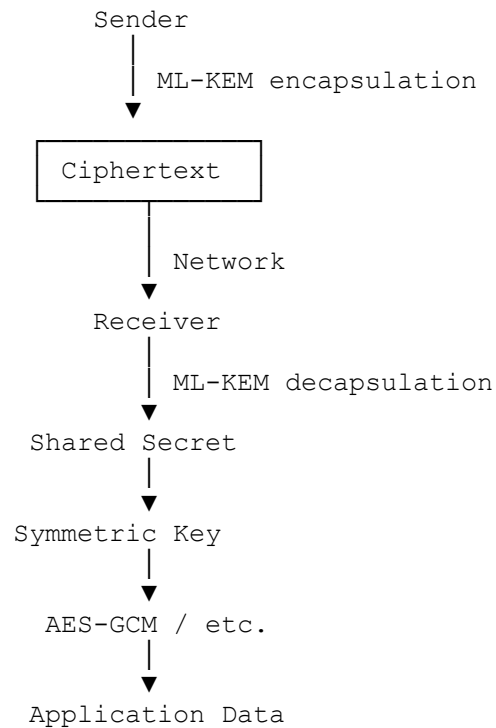
The shared secret can then be used with symmetric encryption:



Data

This is a common **hybrid cryptographic architecture**.

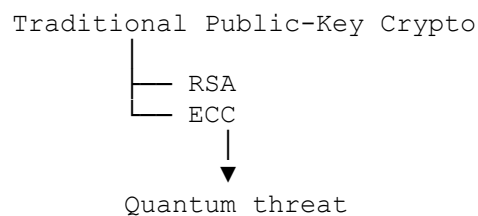
Complete Secure-Communication Model

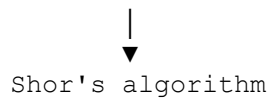


Why Is ML-KEM "Quantum Resistant"?

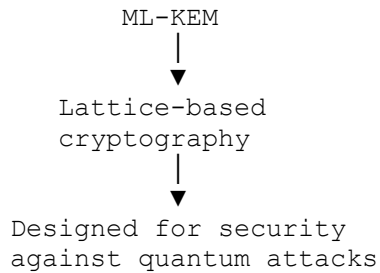
The security of ML-KEM is based on difficult mathematical problems involving **structured lattices**, rather than the integer-factorization/discrete-log problems underlying RSA and traditional elliptic-curve cryptography.

Conceptually:





Whereas:

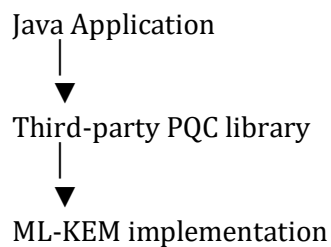


"Quantum-resistant" does **not** mean mathematically proven impossible to break with any future quantum computer. It means the algorithm is designed around problems currently believed to remain hard for quantum computers.

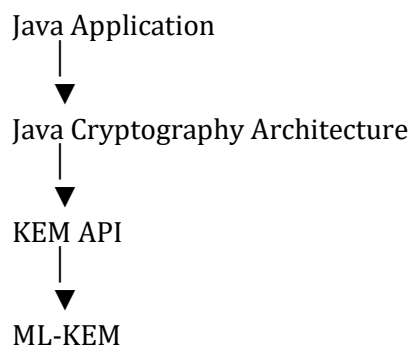
Why Java Added ML-KEM

Java applications increasingly need post-quantum cryptography.

Without a standard API:



Java 24 provides a standardized cryptographic abstraction:



This makes post-quantum key establishment more accessible to Java applications and frameworks.

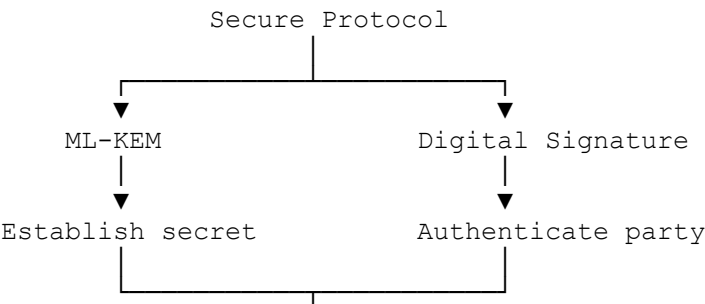
ML-KEM vs RSA/ECDH

Property	RSA	ECDH	ML-KEM
Purpose	Encryption/signatures	Key agreement	Key encapsulation
Classical security	Yes	Yes	Yes
Designed for quantum resistance	✗	✗	✓
Mathematical basis	Integer factorization	Elliptic curves	Lattices
Java 24 standard KEM support	—	—	✓
Suitable for PQC migration	✗	✗	✓

ML-KEM does **not** replace digital signatures such as RSA or ECDSA by itself. For a complete post-quantum system, authentication/signature algorithms also need consideration.

Important Security Distinction

ML-KEM provides **key establishment**, not identity authentication by itself.

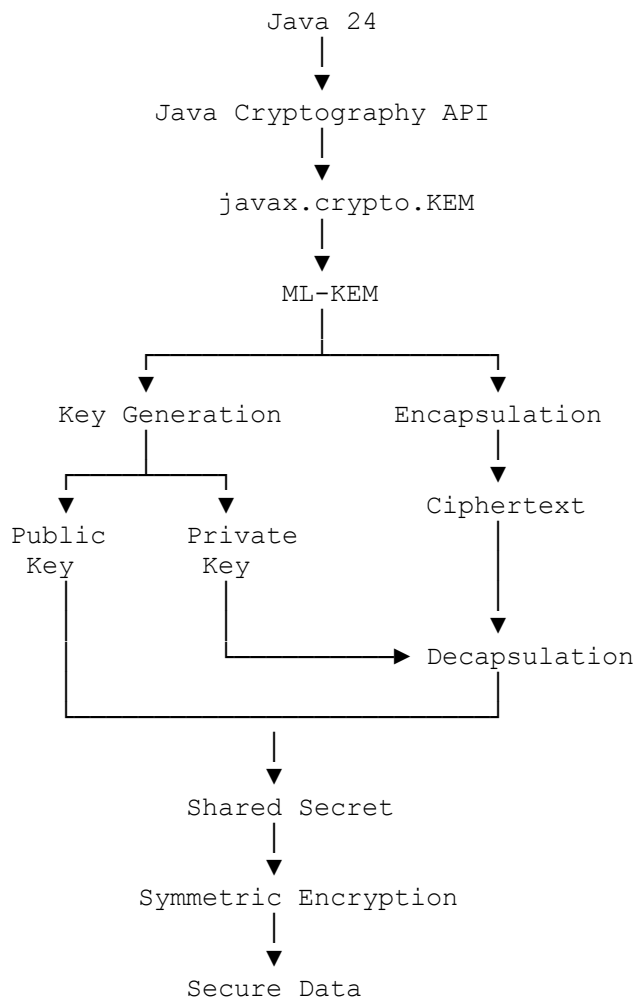


▼
Secure Session

Therefore, a real production protocol may need both:

- A KEM for establishing encryption keys
 - Authentication/signatures for proving who the communicating party is
-

Overall Architecture



Key Takeaways

Property	Details
----------	---------

Java Version	24
Year	2025
JEP	496
Status	Final
Feature	Quantum-Resistant Module-Lattice-Based Key-Encapsulation Mechanism
Algorithm	ML-KEM
API	javax.crypto.KEM
Purpose	Post-quantum key establishment
Security basis	Module lattices
Standard parameter sets	ML-KEM-512, ML-KEM-768, ML-KEM-1024
Java syntax changes	None
Produces	Ciphertext + shared secret
Requires private key	For decapsulation
Large-data encryption	Normally use derived symmetric keys instead
Digital signatures	ML-KEM does not provide authentication by itself

One-line definition

Java 24's Quantum-Resistant ML-KEM (JEP 496) adds standardized ML-KEM key-encapsulation support to the Java Cryptography Architecture, enabling applications to establish shared secrets using a lattice-based algorithm designed to withstand attacks from both classical and future quantum computers.

Java 24 — Quantum-Resistant ML-DSA

Java Version: 24

Year: 2025

Feature: Quantum-Resistant ML-DSA

JEP: 497

UniqueClassName: QuantumResistantMLDSA

Definition

Quantum-Resistant ML-DSA adds standardized support for **ML-DSA (Module-Lattice-Based Digital Signature Algorithm)** to the Java Cryptography Architecture (JCA).

ML-DSA is a **post-quantum digital signature algorithm** designed to provide authentication and integrity protection against both classical computers and sufficiently powerful quantum computers.

Java 24 adds ML-DSA support through the standard Java security APIs.

The primary APIs include:

`java.security.KeyPairGenerator`

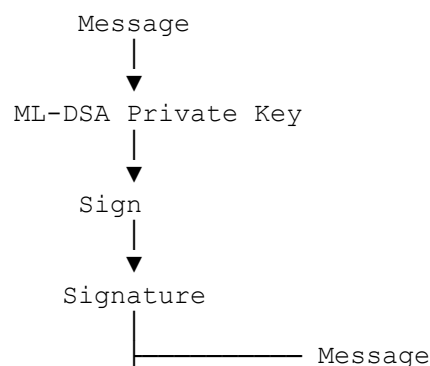
`java.security.Signature`

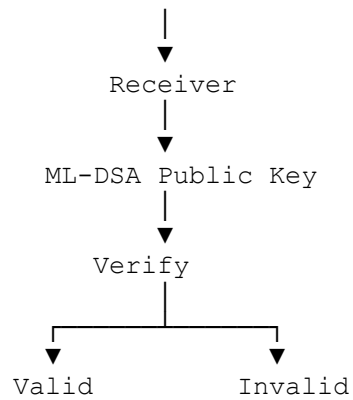
with the algorithm name:

"ML-DSA"

What Does ML-DSA Do?

ML-DSA allows a sender to digitally sign data.





The private key is used to **create** the signature.

The public key is used to **verify** it.

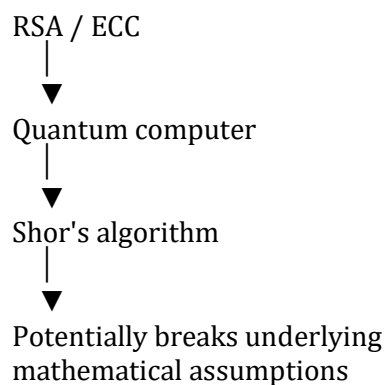
Why Is ML-DSA Quantum Resistant?

Traditional digital-signature algorithms include:

RSA
ECDSA
EdDSA

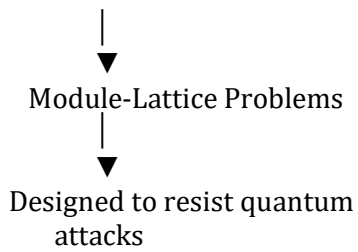
Their security relies on mathematical problems that could be threatened by sufficiently capable quantum computers.

For example:



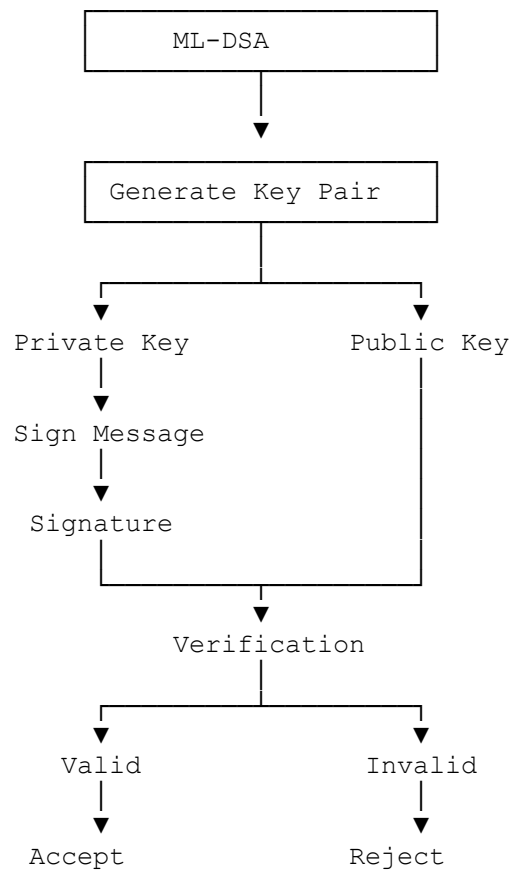
ML-DSA instead uses a **module-lattice-based** construction.

ML-DSA



"Quantum-resistant" means the algorithm is designed using mathematical problems believed to remain difficult for quantum computers; it does not mean that future cryptanalysis can never find weaknesses.

Flowchart



Java Code Example — Generate ML-DSA Keys

Java 24 can generate an ML-DSA key pair using `KeyPairGenerator`.

```
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.Security;

public class QuantumResistantMLDSA {

    public static void main(String[] args)
        throws Exception {

        KeyPairGenerator generator =
            KeyPairGenerator.getInstance("ML-DSA");

        KeyPair keyPair =
            generator.generateKeyPair();

        System.out.println(
            "ML-DSA key pair generated."
        );

        System.out.println(
            "Public key algorithm: "
            + keyPair.getPublic().getAlgorithm()
        );

        System.out.println(
            "Private key algorithm: "
            + keyPair.getPrivate().getAlgorithm()
        );
    }
}
```

Output is conceptually:

```
ML-DSA key pair generated.
Public key algorithm      : ML-DSA
Private key algorithm     : ML-DSA
```

Java Code Example — Sign and Verify

A complete example:

```
import java.nio.charset.StandardCharsets;
import java.security.KeyPair;
import java.security.KeyPairGenerator;
```

```

import java.security.Signature;
import java.util.Base64;

public class QuantumResistantMLDSA {

    public static void main(String[] args)
        throws Exception {

        // 1. Generate ML-DSA key pair
        KeyPairGenerator generator =
            KeyPairGenerator.getInstance("ML-DSA");

        KeyPair keyPair =
            generator.generateKeyPair();

        // 2. Message
        byte[] message =
            "Hello, Post-Quantum Java!"
                .getBytes(StandardCharsets.UTF_8);

        // 3. Create signature
        Signature signer =
            Signature.getInstance("ML-DSA");

        signer.initSign(
            keyPair.getPrivate()
        );

        signer.update(message);

        byte[] signature =
            signer.sign();

        System.out.println(
            "Signature created."
        );

        System.out.println(
            "Signature: "
            + Base64.getEncoder()
                .encodeToString(signature)
        );

        // 4. Verify signature
        Signature verifier =
            Signature.getInstance("ML-DSA");

        verifier.initVerify(
            keyPair.getPublic()
        );

        verifier.update(message);

        boolean valid =
            verifier.verify(signature);

        System.out.println(

```

```

        "Signature valid: " + valid
    );
}
}

```

Output will be similar to:

```

Signature created.
Signature: <Base64 signature>
Signature valid: true

```

How the Code Works

Step 1 — Generate Key Pair

```

KeyPairGenerator generator =
    KeyPairGenerator.getInstance("ML-DSA");

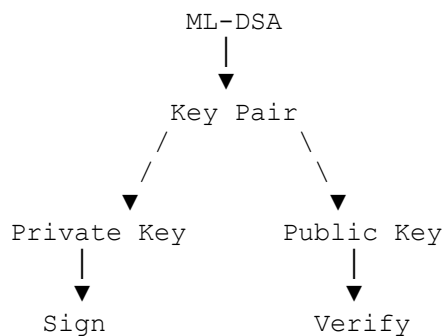
```

```

KeyPair keyPair =
    generator.generateKeyPair();

```

This creates:



The private key must remain secret.

The public key can be distributed to parties that need to verify signatures.

Step 2 — Prepare Message

```
byte[] message =  
    "Hello, Post-Quantum Java!"  
    .getBytes(StandardCharsets.UTF_8);
```

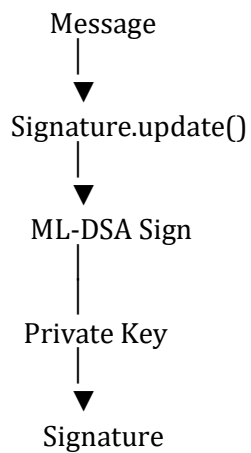
The message could be:

Hello, Post-Quantum Java!

Step 3 — Create Signature

```
Signature signer =  
    Signature.getInstance("ML-DSA");  
  
signer.initSign(  
    keyPair.getPrivate()  
);  
  
signer.update(message);  
  
byte[] signature =  
    signer.sign();
```

Flow:



Step 4 — Verify Signature

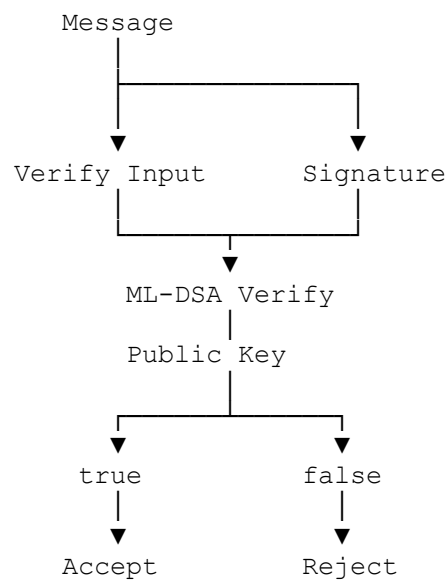
The receiver obtains:

Message
+
Signature
+
Public Key

Then:

```
Signature verifier =  
    Signature.getInstance("ML-DSA");  
  
verifier.initVerify(  
    keyPair.getPublic()  
);  
  
verifier.update(message);  
  
boolean valid =  
    verifier.verify(signature);
```

Flow:

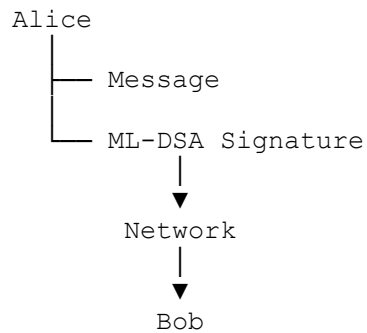


What Does the Signature Protect?

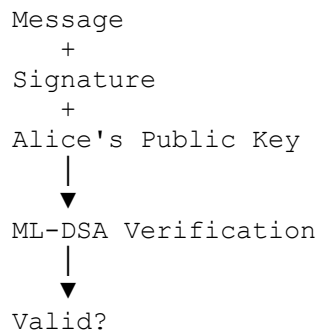
Suppose Alice sends:

Message:
"Transfer ₹10,000"

and signs it.



Bob verifies:

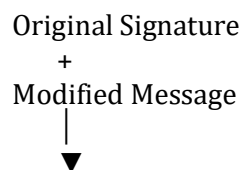


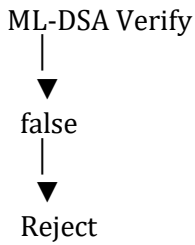
If an attacker changes the message:

Original:
Transfer ₹10,000

Tampered:
Transfer ₹90,000

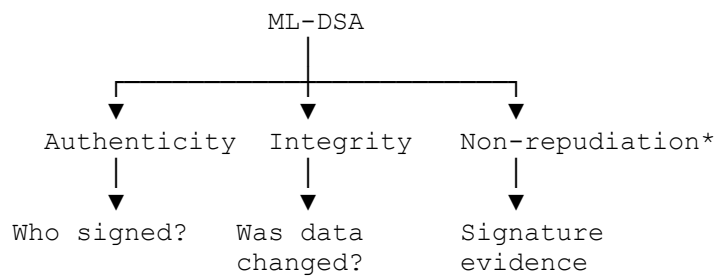
the signature verification should fail:





ML-DSA Security Properties

A digital signature provides important properties:



*The legal/organizational meaning of non-repudiation depends on the broader system and key-management process; a cryptographic signature alone does not establish legal non-repudiation.

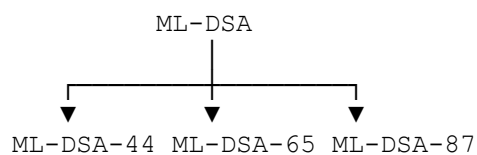
ML-DSA Parameter Sets

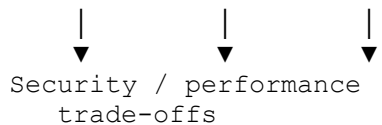
ML-DSA has standardized parameter sets:

ML-DSA-44
ML-DSA-65
ML-DSA-87

They represent different security/performance trade-offs.

Conceptually:





Java's API allows the parameter set to be selected when supported by the provider.

For example:

```
KeyPairGenerator generator =  
    KeyPairGenerator.getInstance("ML-DSA");
```

The exact initialization mechanism should follow the Java 24 API/provider behavior and the application's security requirements.

ML-DSA vs ML-KEM

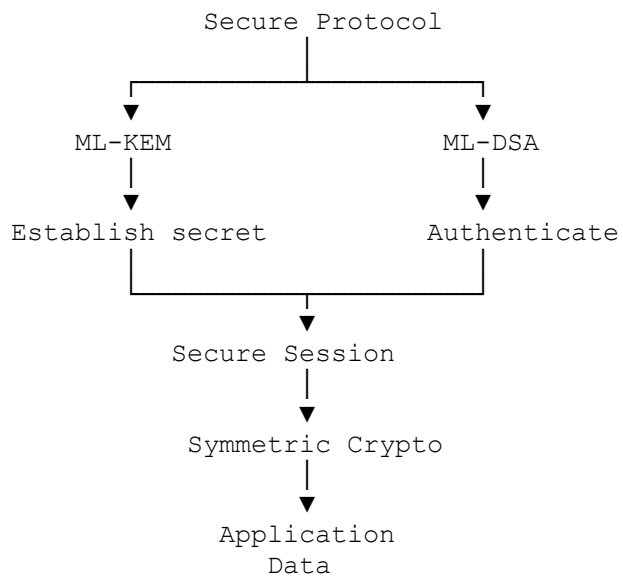
These two Java 24 post-quantum features have different purposes.

Feature	ML-KEM	ML-DSA
JEP	496	497
Purpose	Key establishment	Digital signatures
Main API	javax.crypto.KEM	java.security.Signature
Private key	Decapsulation	Signing
Public key	Encapsulation	Verification
Produces	Shared secret + ciphertext	Digital signature
Protects confidentiality	Indirectly	No
Protects integrity	Indirectly in protocols	✓
Authentication	Not by itself	✓
Post-quantum design	✓	✓

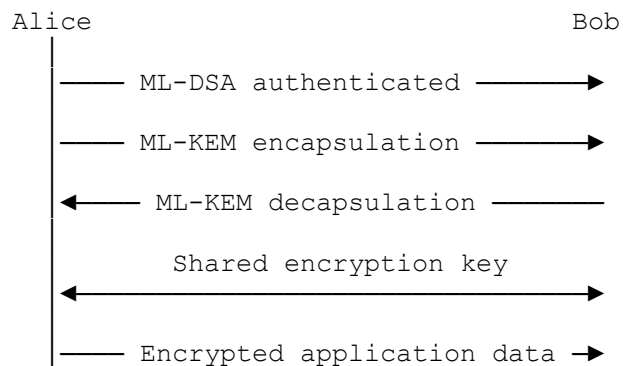
The two algorithms complement each other.

ML-KEM + ML-DSA

A future-resistant secure communication protocol can conceptually combine them:



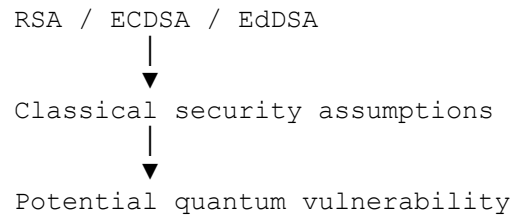
For example:



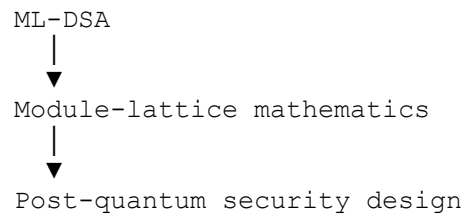
A real protocol should use standardized protocol specifications rather than simply combining primitives ad hoc.

Traditional Signature vs ML-DSA

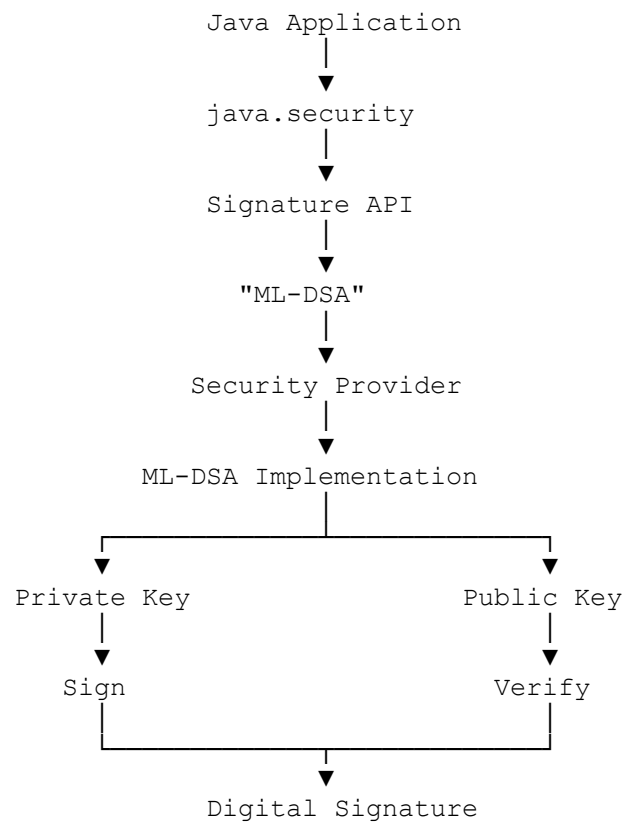
Traditional



ML-DSA

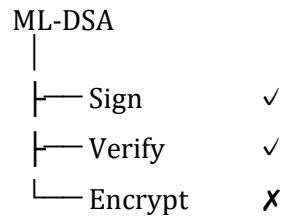


Java 24 API Architecture

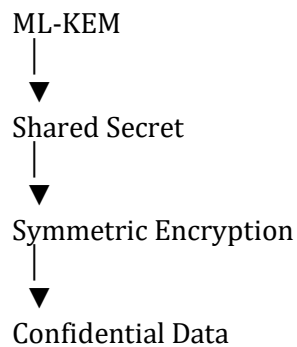


Important Security Note

ML-DSA is a **digital signature** algorithm. It does not encrypt the message.



For confidentiality, a protocol would typically use a key-establishment mechanism such as ML-KEM and then symmetric encryption.



Compilation

ML-DSA support is part of the Java 24 security APIs, so normal Java compilation is used:

```
javac QuantumResistantMLDSA.java
```

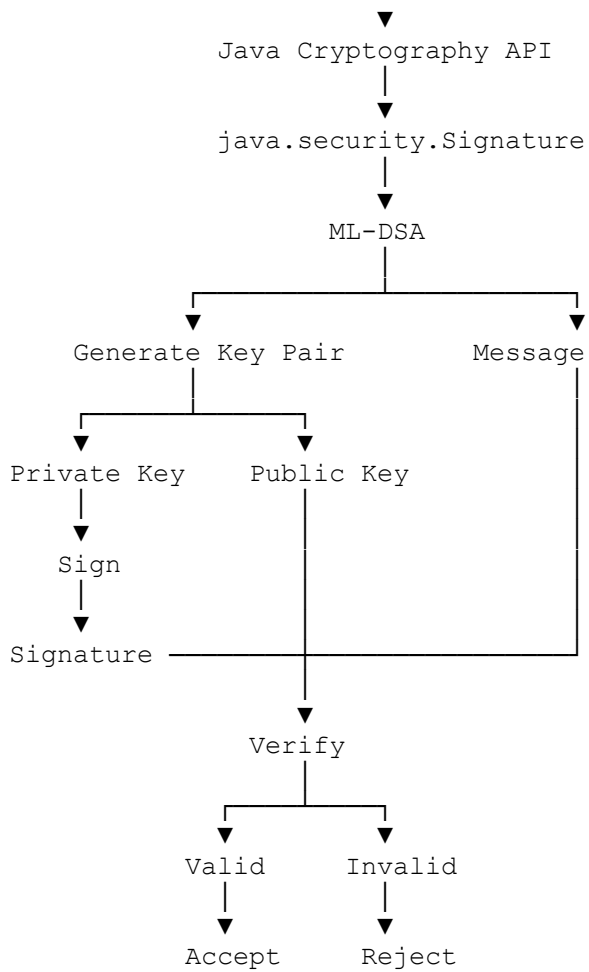
Run:

```
java QuantumResistantMLDSA
```

No `--enable-preview` flag is required for ML-DSA because **JEP 497 is a final Java 24 feature**, not a preview feature.

Overall Architecture

Java 24
|



Key Takeaways

Property	Details
Java Version	24
Year	2025
JEP	497
Status	Final
Feature	Quantum-Resistant Module-Lattice-Based Digital Signature Algorithm
Algorithm	ML-DSA
Primary API	<code>java.security.Signature</code>

Key generation	KeyPairGenerator
Purpose	Post-quantum digital signatures
Security basis	Module-lattice cryptography
Standard parameter sets	ML-DSA-44, ML-DSA-65, ML-DSA-87
Private key	Creates signatures
Public key	Verifies signatures
Encryption	No
Authentication	Yes, when keys are appropriately authenticated/bound to identities
Integrity	Yes
Preview flag	Not required

One-line definition

Java 24's Quantum-Resistant ML-DSA (JEP 497) adds standardized support for the Module-Lattice-Based Digital Signature Algorithm, enabling Java applications to create and verify digital signatures using a post-quantum cryptographic algorithm designed to resist both classical and quantum attacks.

Java 24 — Structured Concurrency (Fourth Preview)

Java Version: 24

Year: 2025

Feature: Structured Concurrency (Fourth Preview)

JEP: 499

UniqueClassName: StructuredConcurrency

Definition

Structured Concurrency is a Java concurrency API that treats a group of related concurrent tasks as a **single unit of work**.

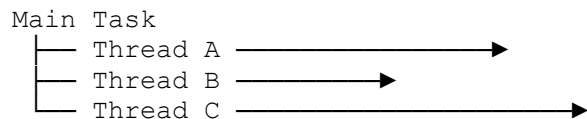
In Java 24, Structured Concurrency is a **fourth-preview feature**. It builds on virtual threads and provides a structured way to:

- Fork multiple concurrent subtasks
- Wait for them as a group
- Propagate failures
- Cancel unfinished subtasks
- Keep task lifetimes within a well-defined scope

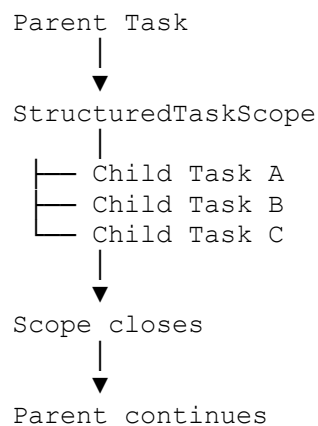
The central idea is:

Concurrent tasks should have a clear owner, lifetime, and completion boundary.

Instead of launching independent threads and manually managing them:



Structured concurrency creates a hierarchy:



Why Structured Concurrency?

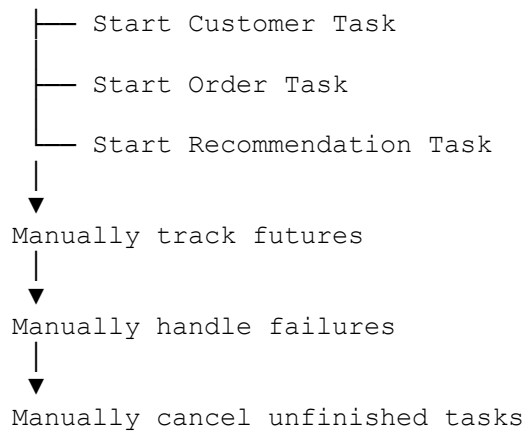
Suppose an application needs three pieces of information:

Customer Service
Order Service
Recommendation Service

They can be queried concurrently.

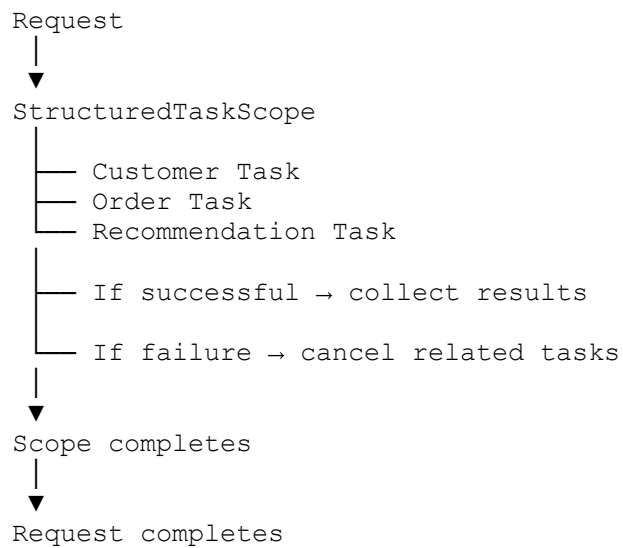
Without structured concurrency:

Request
|



This can become difficult to reason about.

Structured concurrency gives the tasks a common scope:

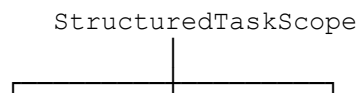


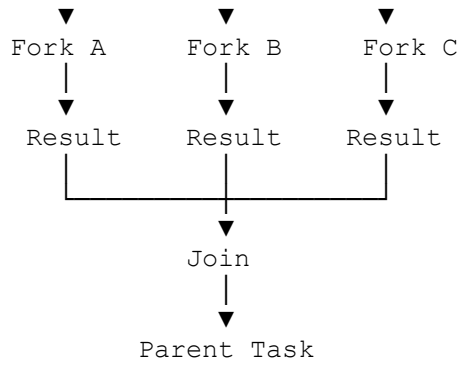
Core Concept

The API is centered around:

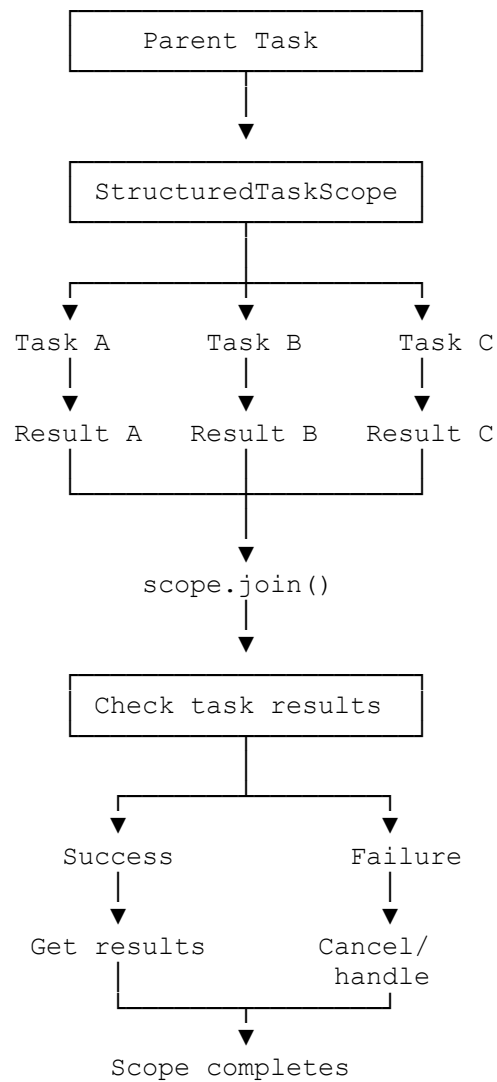
StructuredTaskScope

A simplified model is:





Flowchart



|
▼
Parent continues

Java Code Example

A simple Java 24 example:

```
import java.util.concurrent.StructuredTaskScope;

public class StructuredConcurrency {

    public static void main(String[] args)
        throws Exception {

        try (var scope =
            StructuredTaskScope.open()) {

            var task1 = scope.fork(() -> {
                Thread.sleep(500);
                return "Customer Data";
            });

            var task2 = scope.fork(() -> {
                Thread.sleep(300);
                return "Order Data";
            });

            var task3 = scope.fork(() -> {
                Thread.sleep(700);
                return "Recommendation Data";
            });

            scope.join();

            System.out.println(task1.get());
            System.out.println(task2.get());
            System.out.println(task3.get());
        }
    }
}
```

The three tasks execute concurrently.

Possible output:

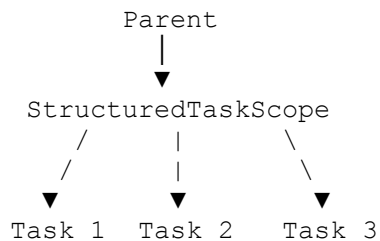
Customer Data
Order Data
Recommendation Data

Understanding the Code

1. Create a Structured Task Scope

```
try (var scope =  
    StructuredTaskScope.open()) {
```

The scope becomes the **parent** of the tasks created inside it.



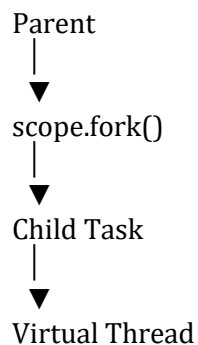
The try block ensures that the scope is properly closed.

2. Fork a Task

```
var task1 = scope.fork() -> {  
    Thread.sleep(500);  
    return "Customer Data";  
});
```

`fork()` starts a concurrent subtask.

Conceptually:

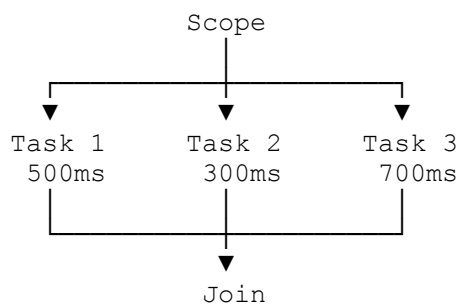


Structured concurrency is designed to work particularly well with **virtual threads**.

3. Fork Multiple Tasks

```
var task2 = scope.fork(() -> {  
    Thread.sleep(300);  
    return "Order Data";  
});  
  
var task3 = scope.fork(() -> {  
    Thread.sleep(700);  
    return "Recommendation Data";  
});
```

Now:



The parent does not need to execute them sequentially.

4. Join the Tasks

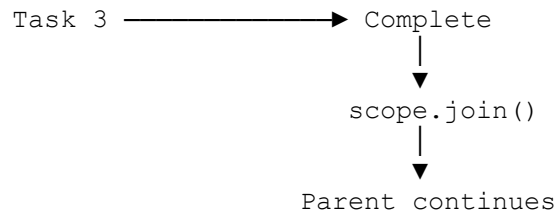
```
scope.join();
```

This means:

Wait until the subtasks in the scope have reached the appropriate completion state.

Conceptually:

```
Task 1 —————> Complete  
Task 2 —————> Complete
```

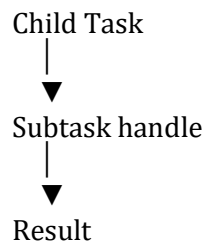


5. Retrieve Results

After joining:

```
System.out.println(task1.get());  
System.out.println(task2.get());  
System.out.println(task3.get());
```

Each forked task provides a handle through which its result can be obtained.



Sequential vs Structured Concurrency

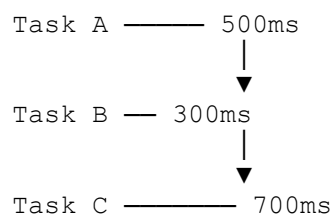
Suppose each operation takes:

Task A = 500 ms

Task B = 300 ms

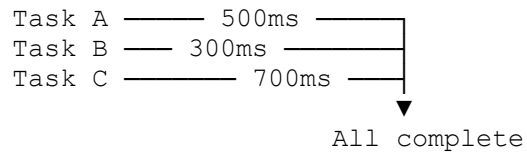
Task C = 700 ms

Sequential



Total $\approx$ 1500ms

Concurrent



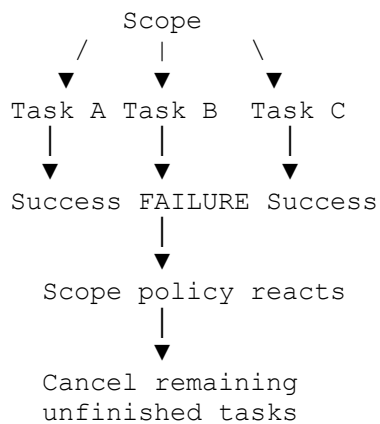
Total $\approx$ 700ms

The exact performance depends on the workload, CPU availability, blocking behavior, and other factors.

Failure Handling

One of the major benefits of structured concurrency is **coordinated failure handling**.

Suppose:



A structured task scope can define a policy for what should happen when one or more subtasks fail.

Shutdown-On-Failure Example

Java's structured concurrency API provides scope configurations that can cancel sibling tasks when a failure occurs.

A representative pattern is:

```
import java.util.concurrent.StructuredTaskScope;

public class StructuredConcurrency {

    public static void main(String[] args)
        throws Exception {

        try (var scope =
            StructuredTaskScope
                .open(
                    StructuredTaskScope
                        .Joiner
                            .allSuccessfulOrThrow()
                )) {

            var task1 = scope.fork(() -> {
                return loadCustomer();
            });

            var task2 = scope.fork(() -> {
                throw new RuntimeException(
                    "Order service failed"
                );
            });

            var task3 = scope.fork(() -> {
                Thread.sleep(2000);
                return loadRecommendations();
            });

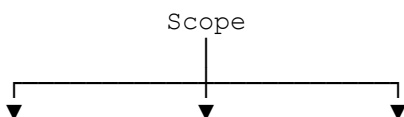
            scope.join();

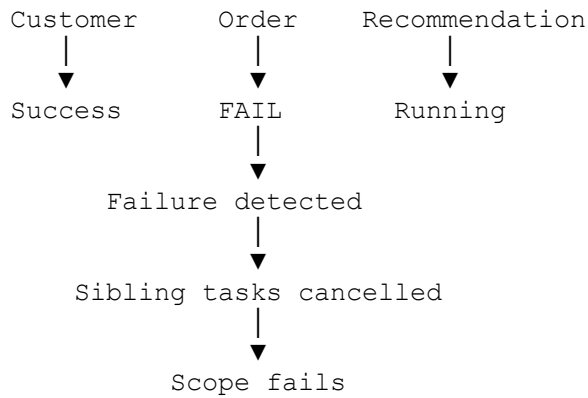
            System.out.println(task1.get());
            System.out.println(task2.get());
            System.out.println(task3.get());
        }

        static String loadCustomer() {
            return "Customer";
        }

        static String loadRecommendations() {
            return "Recommendations";
        }
    }
}
```

The important concept is:

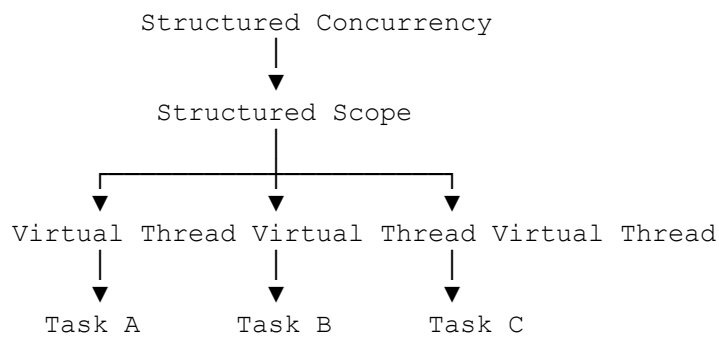




The exact API syntax should be checked against the Java 24 preview API when compiling, because this feature is still in preview and has evolved across JDK releases.

Structured Concurrency and Virtual Threads

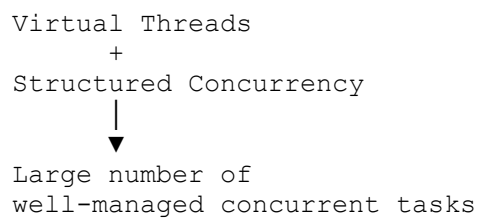
Structured concurrency and virtual threads complement each other.



Virtual threads provide **cheap concurrency**.

Structured concurrency provides **structured lifecycle management**.

Together:



Unstructured vs Structured Concurrency

Unstructured

```
ExecutorService executor =  
    Executors.newVirtualThreadPerTaskExecutor();
```

```
Future<String> a =  
    executor.submit(() -> loadA());
```

```
Future<String> b =  
    executor.submit(() -> loadB());
```

```
Future<String> c =  
    executor.submit(() -> loadC());
```

```
// Manually coordinate futures
```

The developer has to reason about:

- Executor lifetime
 - Future lifetime
 - Cancellation
 - Exceptions
 - Shutdown
 - Parent-child relationships
-

Structured

```
try (var scope =  
    StructuredTaskScope.open()) {
```

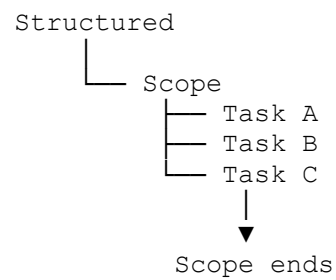
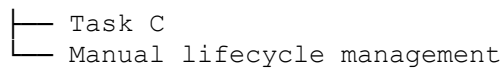
```
    var a = scope.fork(() -> loadA());  
    var b = scope.fork(() -> loadB());  
    var c = scope.fork(() -> loadC());
```

```
    scope.join();
```

```
    // Process results  
}
```

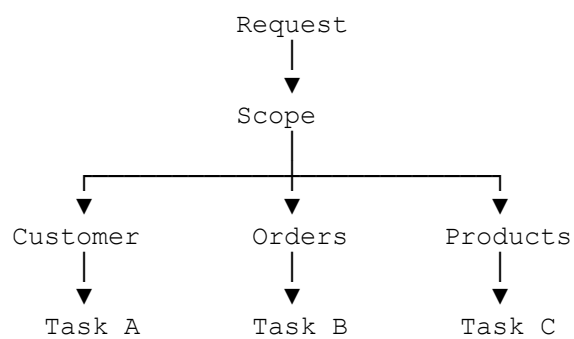
The structure is much clearer:

```
Unstructured  
├── Task A  
└── Task B
```



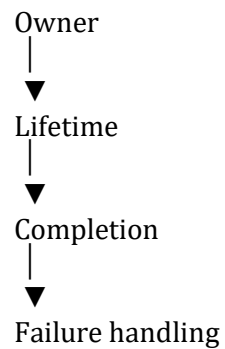
Parent-Child Relationship

Structured concurrency creates a logical task hierarchy.



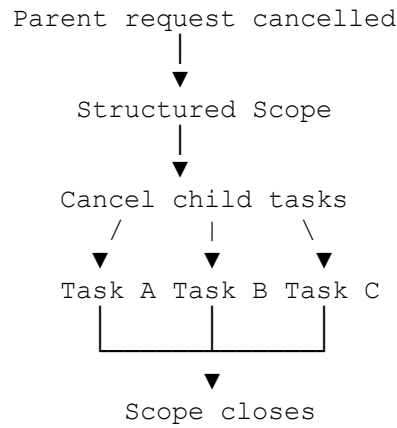
The parent owns the child tasks.

This gives each concurrent task a clear:



Cancellation

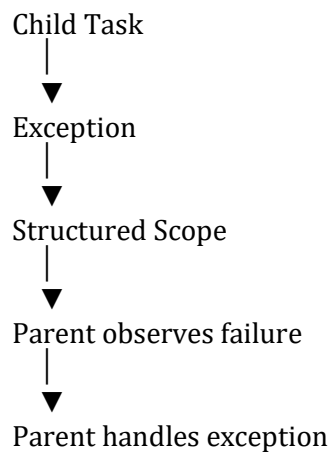
Suppose the parent no longer needs the results:



This is much easier to reason about than independent background tasks that may continue running after their initiating operation has finished.

Exception Propagation

Structured concurrency also establishes a clearer failure boundary.



Instead of an exception becoming detached from the operation that created the task, it remains associated with the structured task hierarchy.

Real-World Example

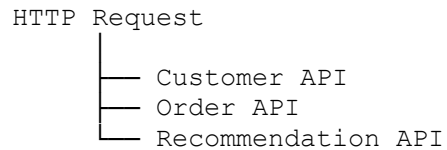
Imagine an HTTP request that needs:

Customer information

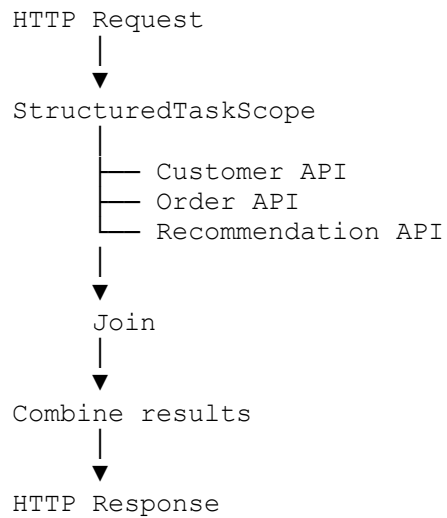
Order history

Product recommendations

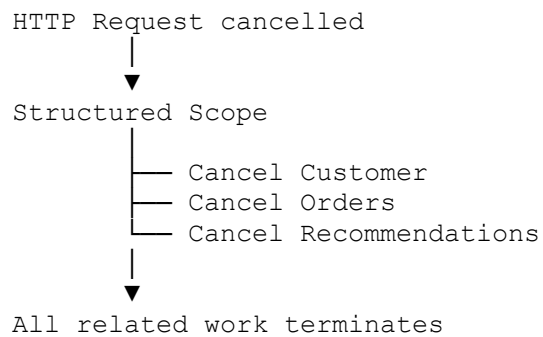
A traditional implementation may look like:



Structured concurrency:



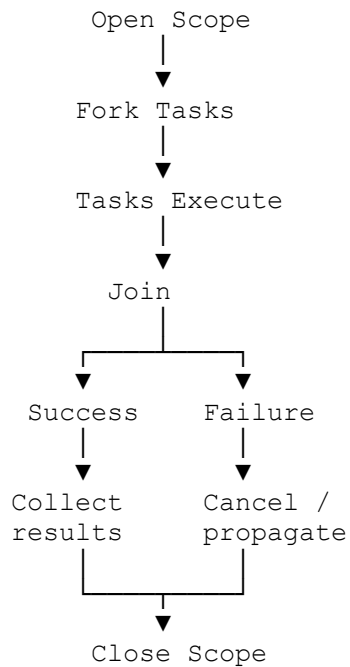
If the request is cancelled:



This is the fundamental structured-concurrency model.

API Lifecycle

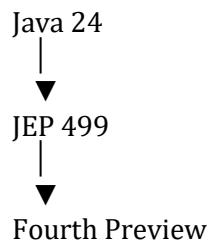
A typical structured-concurrency lifecycle is:



Important: Preview Feature

Structured Concurrency is **not final in Java 24**.

It is:



Therefore, code using it requires preview support.

Compile:

```
javac --enable-preview --release 24 StructuredConcurrency.java
```

Run:

```
java --enable-preview StructuredConcurrency
```

Important API Evolution

Structured Concurrency has changed across its preview iterations.

The general model has remained:

```
StructuredTaskScope
├── fork()
├── join()
└── coordinated completion
```

But specific factory methods, joiner policies, and result APIs have evolved.

Therefore, when writing Java 24 code, use the **Java 24 version of the preview API**, not examples written for Java 21, 22, or 23.

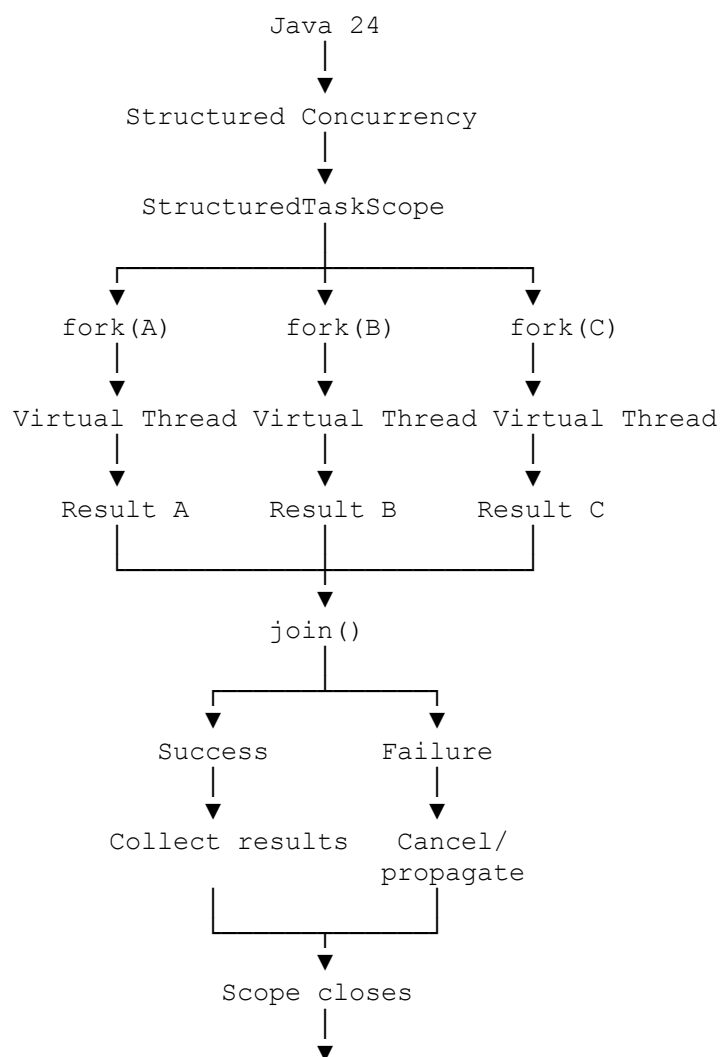
Structured Concurrency vs CompletableFuture

Feature	CompletableFuture	Structured Concurrency
Asynchronous computation	✓	✓
Task hierarchy	Not inherently structured	✓
Parent-child lifetime	Manual	Structured
Cancellation coordination	Manual	Built into scope model
Failure coordination	Manual	Scope-based

Virtual-thread friendly	✓	✓
Main abstraction	Future pipeline	Task scope
Best fit	Async pipelines/composition	Related concurrent subtasks

They are not necessarily competitors. `CompletableFuture` remains useful for asynchronous composition, while structured concurrency is particularly useful when several tasks form one logical operation.

Overall Architecture



Parent continues

Key Takeaways

Property	Details
Java Version	24
Year	2025
JEP	499
Status	Fourth Preview
Feature	Structured Concurrency
Main API	StructuredTaskScope
Purpose	Manage related concurrent tasks as one unit
Task creation	scope.fork(...)
Synchronization	scope.join()
Failure handling	Scope-based
Cancellation	Coordinated
Virtual Threads	Works particularly well with them
Task hierarchy	Parent → child
Java syntax	API-based
Preview flag	--enable-preview

One-line definition

Java 24's Structured Concurrency (JEP 499) provides a fourth-preview API for treating related concurrent tasks as a single structured unit, giving them a common scope, lifecycle, failure-handling policy, cancellation behavior, and completion boundary—especially complementing Java's virtual threads.

Java 24 — Deprecate 32-bit x86 Port for Removal

Java Version: 24

Year: 2025

Feature: Deprecate 32-bit x86 Port for Removal

JEP: ?

There is an important distinction here: **Java 24's JEP 501 is "Deprecate the 32-bit x86 Port for Removal."** This is a **JDK/platform maintenance change**, not a new Java language or library API feature.

UniqueClassName: DeprecateX86Port

Definition

Deprecate the 32-bit x86 Port for Removal means that Java 24 formally **deprecated the HotSpot/JDK port for 32-bit x86 (x86-32) with the intention of removing it in a future release.**

The affected architecture is commonly associated with:

32-bit x86
i386 / i486 / i586 / i686

The important distinction is:

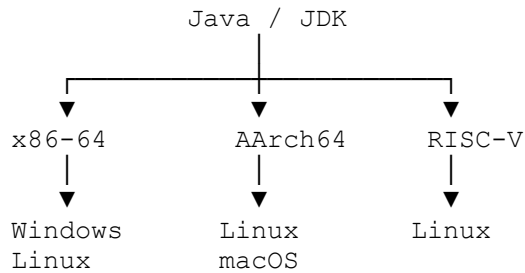
32-bit x86
↓
▼
Deprecated for removal
↓
▼
Future JDK
↓
▼
Port removed

This does **not** mean that Java 24 removes 32-bit x86 immediately. Java 24 begins the deprecation process.

What Is a Java Port?

A **port** is a version of the JDK adapted to a particular operating-system and CPU architecture combination.

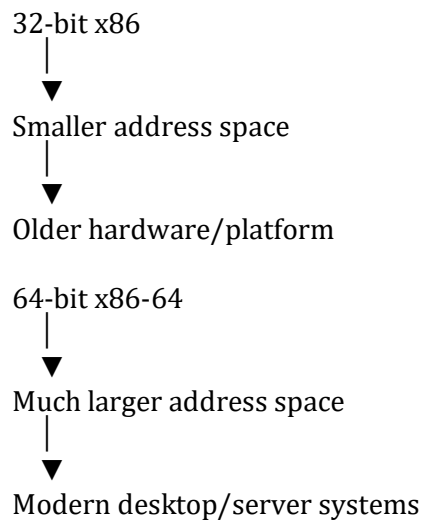
For example:



A 32-bit x86 port specifically supports execution on 32-bit x86 processors.

32-bit vs 64-bit

The distinction is:

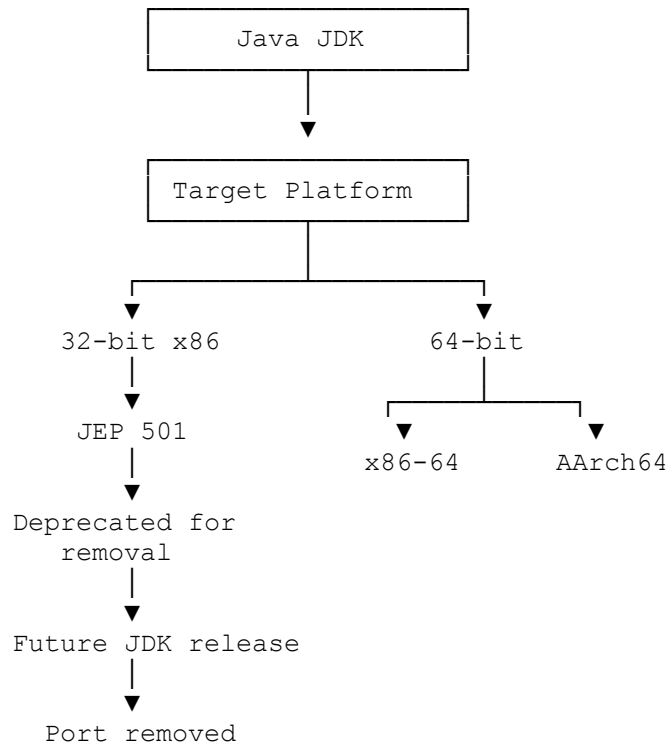


The JEP concerns **32-bit x86**, not modern 64-bit x86-64 systems.

So:

x86-32 → affected
x86-64 → not affected by this JEP

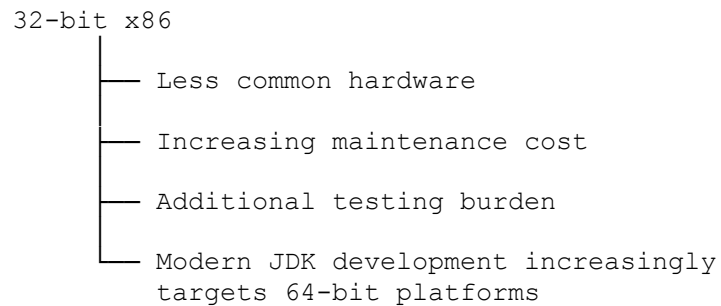
Flowchart



Why Deprecate the 32-bit x86 Port?

The main reason is that **32-bit x86 systems have become increasingly uncommon**, while maintaining a separate JDK port has ongoing engineering and testing costs.

Conceptually:



↓
▼
Deprecate for removal

This allows JDK development resources to focus on architectures that are more widely used.

Java Code Example

Because this is a **platform/JDK-port change**, there is **no new Java language syntax or API** to demonstrate.

A normal Java program continues to work exactly the same way on supported platforms:

```
public class DeprecateX86Port {  
    public static void main(String[] args) {  
        System.out.println(  
            "Java Version: "  
            + System.getProperty("java.version")  
        );  
        System.out.println(  
            "OS Architecture: "  
            + System.getProperty("os.arch")  
        );  
        System.out.println(  
            "Operating System: "  
            + System.getProperty("os.name")  
        );  
    }  
}
```

Possible output on a modern 64-bit machine:

```
Java Version      : 24  
OS Architecture   : amd64  
Operating System  : Windows 11
```

The exact `os.arch` value is implementation/platform dependent.

Detecting the Architecture

You can inspect:

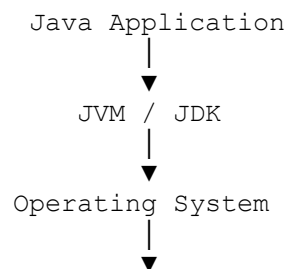
`System.getProperty("os.arch")`

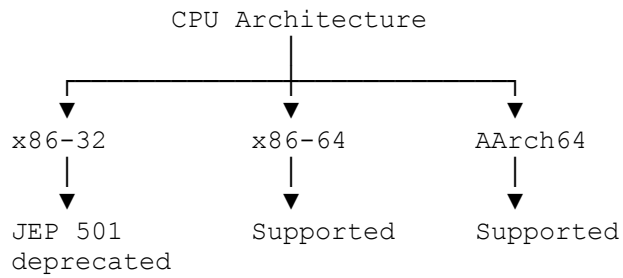
For example:

```
public class DeprecateX86Port {  
    public static void main(String[] args) {  
        String architecture =  
            System.getProperty("os.arch");  
  
        System.out.println(  
            "Architecture: " + architecture  
        );  
  
        if (architecture.equalsIgnoreCase("x86")) {  
            System.out.println(  
                "32-bit x86 architecture detected."  
            );  
        } else {  
            System.out.println(  
                "Not the 32-bit x86 architecture."  
            );  
        }  
    }  
}
```

However, **do not use the string returned by `os.arch` as a universally reliable architecture detector across every JDK/vendor/platform combination.** Its exact value can vary.

Platform Architecture Flow





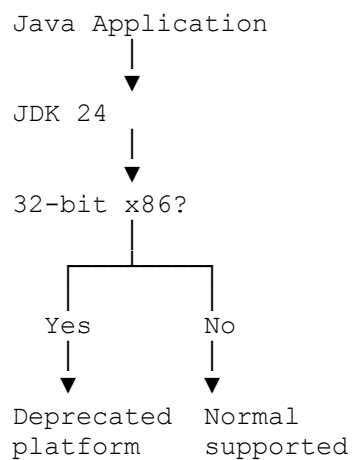
What Happens to Existing Java Code?

Suppose you have:

```
public class Application {  
    public static void main(String[] args) {  
        System.out.println("Application running");  
    }  
}
```

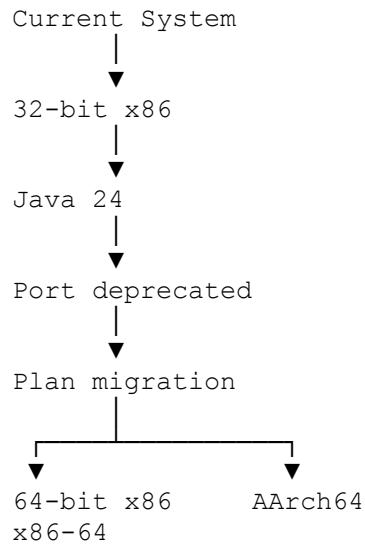
The source code itself does not need to change simply because of JEP 501.

The issue is the **runtime platform**:



Migration Scenario

If an organization still runs Java applications on 32-bit x86:



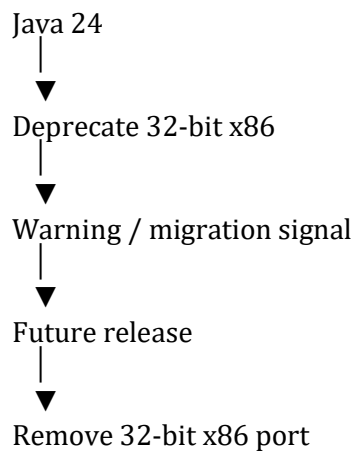
The recommended long-term approach is to move the operating system/application environment to a **supported 64-bit architecture**.

Important Distinction: Deprecation vs Removal

This JEP does **not** mean:

"32-bit x86 support disappears immediately in Java 24."

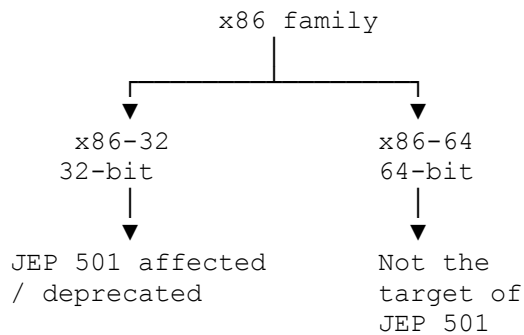
Instead:



This gives users and distributors time to migrate.

32-bit x86 vs x86-64

This is one of the most important points.



A typical modern Intel or AMD desktop running a 64-bit operating system is generally **x86-64**, not the deprecated 32-bit x86 port.

Why 64-bit Matters

Modern Java applications commonly need large amounts of memory.

A 32-bit address space has a fundamentally smaller address range:

32-bit
↓
 2^{32} address values
↓
Much smaller address space

A 64-bit architecture provides:

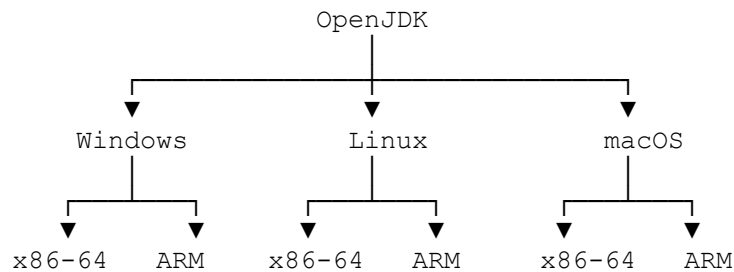
64-bit
↓
 2^{64} theoretical address values
↓

Much larger address space

Actual usable memory is constrained by the operating system, hardware, JVM, and other factors, but the architectural difference is substantial.

JDK Maintenance Perspective

The JDK supports multiple platforms:

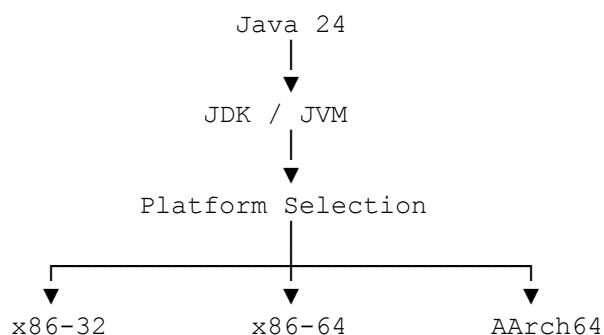


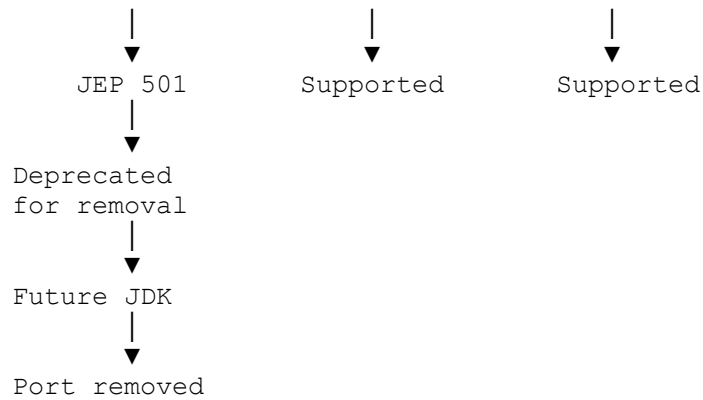
Every additional architecture requires:

- Port-specific code
- Build infrastructure
- Testing
- Bug fixes
- CI resources
- Runtime/JIT maintenance

Removing obsolete architectures reduces this maintenance burden.

Overall Architecture





Key Takeaways

Property	Details
Java Version	24
Year	2025
JEP	501
Feature	Deprecate 32-bit x86 Port for Removal
UniqueClassName	DeprecateX86Port
Type	JDK/platform maintenance
Affected architecture	32-bit x86 / x86-32
x86-64 affected?	No
Java language change	None
New API	None
Java source migration	Normally none
Main impact	Runtime/platform compatibility
Java 24 status	Deprecated for removal
Future direction	Remove the 32-bit x86 port

One-line definition

Java 24's JEP 501 deprecates the 32-bit x86 JDK port for eventual removal, reflecting the declining use of 32-bit x86 systems and allowing JDK development and maintenance to focus on modern 64-bit architectures.